Alexandru Ene Cosmin Ştirbu

Programare orientată pe obiecte

teorie şi aplicaţii în Java

Editura Universităţii din Piteşti
2008

Alexandru Ene, Cosmin Ştirbu

Programare orientată pe obiecte
teorie şi aplicaţii în Java

- Introducere
- Aplicaţii elementare în Java
- Clase
- Vectori. Matrici
- Stringuri
- Moştenirea
- Clase abstracte. Interfeţe
- Polimorfism
- Grafica în Java

Editura Universităţii din Piteşti

Editura Universitãtii din Pitesti

Str. Târgu din Vale, nr.1,
110040, Piteşti, jud. Argeş
tel/fax: 40 248 21.64.48

Copyright © 2008 – Editura Universităţii din Piteşti
Toate drepturile asupra acestei ediţii sunt rezervate
Editurii Universităţii din Piteşti.
Nici o parte din acest volum nu poate fi reprodusă
sub nicio formă, fără permisiunea scrisă a autorului.

Editor: Lector univ.dr. Sorin FIANU
Redactor şef: Conf.univ.dr. Constantin ŢIBRIAN
Bun de tipar: 08.11. 2007; tiraj:150
Referenţi ştiinţifici:
- prof. univ. dr. ing. Ioan Liţă
- prof. univ. dr. ing. Gheorghe Gavriloaia

Descrierea CIP a Bibliotecii Naţionale a României
ENE, ALEXANDRU
Programare orientată pe obiecte . Teorie şi aplicaţii în Java / Alexandru Ene, Cosmin Ştirbu. - Piteşti
Editura Universităţii din Piteşti, 2008
Bibliogr.
Index
ISBN 978-973-690-845-3

I. Ştirbu, Cosmin

004.43 JAVA
004.42.045

PREFAŢĂ

Datorită dezvoltării hard-ului (procesoare si memorii mai performante), programele au putut deveni din ce în ce mai mari (ca număr de linii de cod). Programele mari, conţin mai multe erori , ceea ce implică un cost de fabricaţie mare. Programarea orientata pe obiecte (POO) este o nouă metodologie de programare care conduce la dezvoltarea unor aplicaţii mari, mai fiabile şi într-un timp mai scurt, decât programarea structurată.

Java este un limbaj de POO, aparut in anul 1995, foarte utilizat in dezvoltarea de aplicatii distribuite. Acest limbaj este de asemenea foarte frecvent folosit, datorită clarităţii şi simplităţii lui, în studierea conceptelor specifice de programare orientată pe obiecte.

Cartea se adreseaza studenţilor de la facultăţi tehnice, în scopul învăţării notiunilor de baza ale programării orientate pe obiecte, in limbajul Java. Se presupune cunoscut limbajul C.

Piteşti, 2008

prof. dr. ing. Alexandru Ene
Universitatea din Piteşti

CUPRINS

Prefață	**5**
Cap. 1 Introducere	**7**
Cap. 2 Aplicații elementare în Java	**11**
Cap. 3 Clase	**17**
Cap. 4 Vectori. Matrici	**37**
Cap. 5 Stringuri	**55**
Cap. 6 Moştenirea	**63**
Cap. 7 Clase abstracte. Interfețe	**75**
Cap. 8 Polimorfism	**79**
Cap. 9 Excepții	**89**
Cap. 10 Fişiere	**95**
Cap. 11 Grafica în Java	**105**
Bibliografie	**125**

Capitolul 1
Introducere

1. **Ce este programarea orientată pe obiecte?**
2. **Ce este un obiect?**
3. **Ce este o clasă?**
4. **De ce a apărut?**
5. **Istoric limbaje POO.**
6. **Limbajul Java. Istoric. Avantaje. Dezavantaje.**

1. Ce este programarea orientată pe obiecte?

Programarea orientată pe obiecte (prescurtat: POO) este un stil de programare în care descompunerea aplicației se face plecând de la obiecte şi nu de la funcții, aşa cum se făcea în programarea structurată. Proiectarea unei program ce foloseşte metodologia POO începe cu găsirea obiectelor ce intervin în aplicație, găsirea operațiilor ce se fac asupra acestor obiecte, găsirea claselor şi a relațiilor dintre clase.

2. Ce este un obiect?

Un obiect este un lucru (o entitate soft) caracterizat prin însuşiri şi comportament. Însuşirile sunt modelate prin variabile **(variabile de instanță)**, iar comportamentul prin funcții. În terminologia POO, funcțiile sunt denumite **metode**.

Un obiect este mai mult decât o structură de date, este o entitate ce are o stare şi este caracterizat de operații ce pot fi executate asupra lui.

Exemplu:

Obiectul cerc poate să aibă ca şi însuşiri caracteristice:

- coordonatele cercului: (x0,y0)
- culoarea cu care se desenează
- raza.

Ca operații (metode), ce-i determină comportamentul:

- calculArie()
- calculPerimetru()
- getRaza() - returnează raza
- getX0()
- getY0()
- setRaza() – setează raza

- setX0()
- setY0()

3. Ce este o clasă?

Orice obiect se instanțiază (se crează) dintr-o clasă.

O clasă poate fi văzută ca un **şablon** din care se instanțiază obiecte sau ca o "fabrică" de obiecte.

Clasa poate fi interpretată şi ca o generalizare a noțiunii de tip de date.

În general, o clasă conține variabile de instanță şi metode.

Există clase particulare care nu au decât variabile de instanță şi clase particulare care nu au decât metode.

Clasele particulare ce conțin doar variabile de instanță sunt echivalentul structurilor (din limbajul C) .

În limbajul Java, ca şi în limbajul C++, crearea unui obiect **(instanțiere)** se face cu ajutorul operatorului **new**.

Se recomandă ca numele unei clase să înceapă cu literă mare, iar numele unui obiect (instanțiat dintr-o clasă) să înceapă cu literă mică.

Exemplu:

Avem clasa Cerc şi obiectul c1 din această clasă.

Cerc c1; // declarația unui obiect c1 de tipul Cerc.

De fapt, c1 este numele unei variabile care va ține adresa (referința) unui obiect de tipul Cerc. Obiectul propriu-zis va fi creat în zona de memorie denumită **heap**.

Declarația unui obiect nu crează obiectul. Obiectul este instanțiat cu ajutorul operatorului **new**.

Exemple:

```
c1=new Cerc( );
c2=new Cerc(0,0,20);
```

4. De ce a apărut POO ?

Datorită dezvoltării hard-ului (procesoare si memorii mai performante), programele au putut deveni din ce în ce mai mari (ca număr de linii de cod). Programele mari au mai multe erori latente, ceea ce implică un cost de fabricație mare. POO este un nou stil de programare care conduce la dezvoltarea unor aplicații mari, mai fiabile (mai puține erori) şi într-un timp mai scurt.

De asemenea, cu tehnologia POO se modelează mai bine aplicația (realitatea).

5. Istoric limbaje POO.

În anii '60, la nivel de cercetare este dezvoltat limbajul de programare Simula, primul limbaj ce introduce noțiunea de clasă.

În anii '80 apar două tendințe în limbajele de POO:

1) crearea unor limbaje pur orientate pe obiect
2) adăugarea la limbajele procedurale existente, a suportului pentru POO.

Ca limbaje pur orientate pe obiect:

- limbajul Smalltalk (firma Xerox) , preia noțiunea de clasă de la Simula şi adaugă noi concepte de bază pentru POO, cum ar fi:
  - moştenire
  - legare dinamică (polimorfism)

  Este un limbaj mai ales pentru cercetători.
- Limbajul Eiffel

Limbajul C++, apărut la începutul anilor '80, urmează cea de-a doua tendință în limbajele de POO. Este vechiul limbaj C, la care s-a adăugat suportul pentru POO.

6. Limbajul Java. Istoric. Avantaje. Dezavantaje.

Limbajul Java este un limbaj pur orientat pe obiecte şi a apărut în anii '90.

Istoric: O echipă de la firma Sun Microsystems, a dezvoltat un limbaj de programare nou, portabil, pentru microcontrolere (pentru programarea unor dispozitive inteligente: telecomenzi inteligente, dispozitive electrocasnice inteligente, etc.).

Portabilitatea este caracteristica programului de a fi executat pe mai multe platforme diferite (sisteme de operare diferite şi/sau procesoare diferite). Avem portabilitate la nivel sursă sau la nivel binar. Portabilitatea la nivel sursa este realizată în măsura în care programul sursă conține instrucțiunile şi funcțiile standardizate ale limbajului respectiv. Programul sursă trebuie sa fie recompilat pe noua platformă de lucru.

Portabilitatea la nivel binar a programului Java se obține prin compilarea programului în două etape. Echipa de programatori de la

firma Sun a realizat portabilitatea la nivel binar realizând compilarea de la limbajul sursă **(.java)** pentru un procesor virtual denumit **maşina virtuală Java (JVM – Java Virtual Machine)**. Fişierul compilat pentru acest procesor virtual are extensia **.class**, iar codul rezultat se cheamă **cod de octeţi**.

Este o compilare la nivel maşină pentru o maşină virtuală .

În dispozitivul în care se doreşte execuţia aplicaţiei trebuie să fie implementată maşina virtuală Java şi această maşină, existentă în dispozitivul unde se execută aplicaţia, interpretează codul de octeţi (execută câte o instrucţiune, pe rând) sau codul de octeţi (fişierul .class) poate fi compilat mai departe pentru procesorul real din sistem (compilare just-in-time). Execuţia interpretată este mai lentă, acesta fiind principalul dezavantaj al limbajului Java, comparativ cu C++.

Proiectul a reuşit, dar echipa a fost desfiinţată pe motiv că cererea de dispozitive inteligente a scăzut. Directorul echipei soft nu a renunţat, văzând ca o utilizare deosebită pentru Java dezvoltarea de aplicaţii pentru Internet. Astfel că, în 1995 a făcut o demonstraţie publică şi a dezvoltat un browser scris în Java (Hot Java) care avea capacitatea să descarce dintr-o pagină web şi să execute pe maşina client, mici aplicaţii scrise în Java denumite **applet-uri**. Aceste applet-uri erau înglobate ca şi fişiere .class (fişiere de octeţi) în pagina web. Astfel, pentru prima dată în istoria Internet-ului, paginile web au devenit dinamice (cu animaţie).

Şi astăzi, limbajul Java este foarte utilizat pentru dezvoltarea de aplicaţii distribuite.

Avantaje:

- portabilitate la nivel binar
- are sintaxa limbajului C (80% - 90%)
- este pur orientat pe obiecte
- are aspecte de securitate mai bine puse la punct decât limbajul C
- a eliminat din limbajul C noţiunile controversate (lucrul cu pointeri)
- a eliminat moştenirea multiplă
- suportă lucrul cu fire de executie.
- verifică indexarea tablourilor.
- este distribuit gratuit de firma Sun.

Capitolul 2
Aplicaţii elementare în Java

1. **Prima aplicaţie Java.**
2. **Ce sunt parametrii din linia de comandă?**
3. **Cum se face citirea de la tastatură?**
4. **Pachete de clase.**

1. Prima aplicaţie Java

Orice aplicaţie Java trebuie să conţină o clasă ce are definită metoda **main( )**.

Cel mai simplu program Java: vom afişa un text pe ecran. Aplicaţia va consta dintr-o singură clasă, iar această clasă nu are variabile de instanţă, numai o singură metodă (metoda main( )).

Să afişăm pe ecran urmatorul text, care se găseşte în Biblie: "Căutaţi mai întâi Împărăţia lui Dumnezeu!".

```
class MaiIntai
{
  public static void main(String args[] )
  {
     System.out.println("Cautati mai intai Imparatia lui Dumnezeu!");
  }
}
```

2. Ce sunt parametrii din linia de comandă?

Exemplul 1

Scriem o clasă în care pentru un număr natural dat ca parametru în linia de comandă îi vom afişa divizorii. Dezvoltăm clasa **AfişareDivizori.**

Compilarea este operaţia de traducere din limbajul sursă (.java) în limbajul maşină al maşinii virtuale Java. În urma compilării ⇒ fişierul .class.

javac AfişareDivizori.java <enter>

Execuţia programului se face folosind interpretorul **java** (interpretează programul instrucţiune cu instrucţiune).

Exemplu:

java AfişareDivizori 6 <enter>

- În urma execuției, se afișează: 1
  2
  3
  6

Parametrii din linia de comandă sunt în mod automat memorați ca **String-uri** în vectorul **args[ ]**, ce e dat ca parametru în **metoda main( )**.

Astfel, primul argument din linia de comandă este memorat în args[0].

Numele clasei trebuie să corespundă cu numele fișierului sursa.

```
class AfișareDivizori
{
  public static void main(String args[ ] )
    {
      // în args[0] s-a memorat numărul tastat, ca String!
      // convertesc numărul de la String la int:
       int nr=Integer.parseInt(args[0]);
       int i;
       for (i=1; i<=nr; i++)
              if (nr%i= =0)
                     System.out.println(i);
    }// main
}// class
```

Exemplul 2

Să se calculeze primul număr prim mai mare decât un număr dat ca parametru în linia de comandă.

```
class PrimulNrPrim
{
 public static void main(String args[])
 {
   int nr=Integer.parseInt(args[0]);
  int nrCrt=nr+1;//cautam incepand cu nr+1
  for(;;){
    if(estePrim(nrCrt))
      break;//l-a gasit
    else nrCrt++;
  }
```



```
  System.out.println("Primul numar prim mai mare  este: "+nrCrt);
}
 private static boolean estePrim(int x)
 {
  boolean este=true;
  for(int i=2;i<=Math.sqrt(x);i++)
    if(x % i == 0){
      este=false;
      break; }
   return este;
 }
}
```

Specificatorii **public**, **private** și **static**, vor fi explicați ulterior.

3. Cum se face citirea de la tastatură?

Citirea de la tastatură în Java se poate face în două moduri:

- în mod consolă (text)
- în mod grafic.

Citirea în modul text este mai complicată, Java lucrând cu tastatura, la fel cum lucrează cu un fișier.

Citirea în modul grafic:

Pentru a citi de la tastatură putem folosi clasa: **JOptionPane** și metoda: **showInputDialog( )**

Exemplu:

```
String nume= JOptionPane.showInputDialog("dați un nume");
```

Exemplul 1

Afișarea maximului a trei numere citite de la tastatură.

```
import javax.swing.*;
class Maxim
{
    public static void main(String args[ ])
    {
        String s1= JOptionPane.showInputDialog("nr1=");
        int nr1= Integer.parseInt(s1);
        int nr2=Integer.parseInt(JOptionPane.showInputDialog(
                "nr2="));
        int nr3=Integer.parseInt(JOptionPane.showInputDialog(
```

```
                "nr3="));
        int max=nr1;
        if (nr2>max) max=nr2;
        if (nr3>max) max=nr3;
        System.out.println("Maxim="+max);
    }
}
```

Operatorul "+" este un operator de concatenare şi realizează concatenarea într-un şir a unui şir şi a altui operand(tot şir sau număr întreg, real, etc.).

În privința compilării acestui program:

```
javac Maxim.java
java Maxim              <enter>
nr1=7
nr2=10
nr3=6
Maxim=10
```

Exemplul 2

Se citesc de la tastatură trei numere întregi a, b şi c. Să se ordoneze aceste numere crescătoare, astfel încât a va fi minimul, iar în c va fi maximul. Citirea de la tastatură se va face cu metoda showInputDialog() din clasa JOptionPane.

```
import javax.swing.*;
class Ordonare3
{
 public static void main(String args[])
 {
   int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
   int b=Integer.parseInt(JOptionPane.showInputDialog("b="));
   int c=Integer.parseInt(JOptionPane.showInputDialog("c="));
   if(a>b){
        // le comutam:
    int aux=a; a=b; b=aux;
   }
   if(a>c){
                //le comutam:
                int aux=a; a=c; c=aux;
```



```
   }
   if(b>c){
        //le comutam:
        int aux=b;b=c; c=aux;
   }

   System.out.println(a+" "+b+" "+c);
}
}
```

Exemplul 3

Se citesc două numere a şi b. Să se afişeze dacă cele două numere au acelaşi număr total de divizori.

```
import javax.swing.*;
class Comparare
{
 public static void main(String args[])
 {
   int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
   int b=Integer.parseInt(JOptionPane.showInputDialog("b="));
   int na=2; // numarul de divizori ai lui a
   // orice numar are ca diviyori pe 1 si el insusi
   for(int i=2;i<=a/2;i++)
     if(a%i==0)na++;
    int nb=2; // numarul de divizori ai lui b
    for(int i=2;i<=b/2;i++)
     if(b%i==0)nb++;
   if(na>nb)System.out.println(a+" are mai multi divizori ");
   else if(na==nb)System.out.println("acelasi numar de divizori ");
   else System.out.println(b+" are mai multi divizori ");
}
}
```

4. Pachete de clase.

Clasa **JOptionPane** face parte din pachetul de clase **javax.swing**. Clasele sunt organizate în pachete de clase. S-a adoptat soluția cu organizarea claselor în pachete de clase, în primul rând pentru evitarea

conflictului de nume de clase. Numele complet al unei clase este dat de **numePachet.numeClasă**.

• Exemple de pachete de clase:
- javax.swing (clasele pentru grafica)
- java.lang (clasele de bază ale limbajului: String, Integer, System, etc.)
- java.util (clasa Random)
- java.io (pentru fişiere: FileOutputStream)
- java.sql (pentru baze de date)
- java.net (pentru clase de comunicații în rețea)

Într-o aplicație trebuie folosit numele complet al clasei, adică **Pachet.nume**.

• Exemplu:

String nume=**javax.swing.** JOptionPane.showInputDialog
("dați un nume=");

Pentru a evita folosirea numelui complet al clasei de fiecare dată când se foloseşte într-o aplicație (deoarece este dificil de scris un nume aşa de lung), trebuie ca pachetul din care face parte clasa respectivă să fie în mod explicit **importat** la începutul programului. Exemplu:

import javax.swing.*

sau se poate importa în mod explicit doar clasa respectivă:

import javax.swing.JOptionPane;

Pachetul **java.lang** este în mod implicit importat în orice aplicație java.

Capitolul 3
Clase

1. **Specificatori de acces.**
2. **Modul de accesare a membrilor publici ai unei clase**
3. **Tipuri primare de date.**
4. **Cum se definesc constantele?**
5. **Exemplu de clasă ce are numai date.**
6. **Exemplu de clasă ce are numai metode.**
7. **Polimorfism parametric.**
8. **Exemplu de clasă ce are şi variabile de instanță şi metode.**
9. **Constructori.**

1. Specificatori de acces.

Prin membrii unei clase înțelegem variabile de instanță şi metode. Accesul la aceşti membrii este controlat prin intermediul unor **specificatori / modificatori de acces.**

Avem următorii patru specificatori de acces (trei cuvinte cheie):

1) **private**
2) **public**
3) **protected**
4) default (implicit).

♦ Specificatorul de acces **private**:

Exemplu:
- pentru o variabilă de instanță: **private int raza;**
- pentru o metodă: **private int calcul( )**

O variabilă de instanță privată nu poate fi accesată decât din metode ale clasei respective.

O metodă privată nu poate fi apelată decât de o altă metodă din clasa respectivă.

Deci, membrii **private** ai unei clase nu pot fi accesați din afara clasei respective.

♦ Specificatorul de acces **public**:

Membrii **public** ai unei clase pot fi accesaţi atât din clasa respectivă, cât şi din orice altă clasă.

♦ Specificatorul de acces **protected**:

Este legat de relaţia de moştenire între clase. Va fi studiat ulterior.

♦ Specificatorul de acces implicit (atunci când în faţa declaraţiei unui membru nu este scris nici unul dintre cei trei specificatori de acces anteriori).

Exemplu:

int raza;

- membrii impliciţi pot fi accesaţi din clasa respectivă, dar şi din orice altă clasă care face parte din **acelaşi pachet** cu clasa respectivă.
- membrii cu accesul implicit nu pot fi accesaţi din alte clase ce fac parte din alte pachete, diferite de pachetul clasei respective.

2. Modul de accesare a membrilor publici ai unei clase.

Din afara clasei respective, membrii publici sau cu accesul implicit ai clasei, se accesează în doi paşi, astfel:

1. instanţiez obiect din clasa respectivă:

2. accesez membrul clasei cu sintaxa:

obiect.numeMembruDeAccesat.

Exemplul 1

```
Cerc c=new Cerc( );
int r=c.getRaza( );
```

Exemplul 2

Fie clasele A si B:

```
class A
{
    public int x;
    ...
}

class B
{
    public static void main(String args[ ])
    {
```



```
    // Vrem sa introducem in x (din clasa A) valoarea 7: x← 7
    x=7; → GREŞIT
    A a=new A( );
    a.x=7;
    }
}
```

A doua posibilitate de accesare este valabilă pentru membrii declaraţi statici. Declararea se face cu ajutorul cuvântului cheie **static**

Membrii statici pot fi accesaţi, pe lângă prima modalitate şi astfel:

- nu se mai instanţiază obiect din clasa respectivă, ci se accesează folosind sintaxa:

NumeClasă.numeMembruDeAccesat.

• Exemplu:

Metoda parseInt( ) din clasa Integer, are semnătura:

public static int parseInt(String)

```
int nr=Integer.parseInt("12");
```

Se recomandă ca variabilele de instanţă ale unei clase să fie declarate **private**, iar accesul la ele să se facă prin metode **public** de tipul:

set( ) – pentru a scrie în ele

get( ) – pentru a citi

Deşi se scrie mai mult cod, folosind nişte metode verificate, programul este mai fiabil.

F. Brooks dă următoarea secvenţă de timpi consumaţi în realizarea unor programe mari (cu mii de linii sursă):

$\frac{1}{3}$ din timpul total, pentru design

$\frac{1}{6}$ codare

$\frac{1}{4}$ testare componente (testarea modulară)

$\frac{1}{4}$ testare ansamblu (testarea de integrare)

3. Tipuri primare de date

În Java există următoarele tipuri primare de date:

a. **int**: numere întregi cu semn, reprezentate pe 4 octeţi.

Spre deosebire de limbajul C, în Java nu există modificatorul de tip **unsigned**.

b. **char** pentru date de tip caracter.

O variabilă de tip **char** ocupă 2 octeți.

Constantele caracter în Java sunt reprezentate cu ajutorul codului **Unicode**.

c. **boolean** pentru date ce au două valori: **true** şi **false**.

d. **byte** numere întregi între -128 şi +127, ocupă un octet.

e. **float** pentru reprezentarea numerelor reale.

f. **double** pentru reprezentarea numerelor reale (precizie mai mare ca tipul **float**).

La instanțierea unui obiect dintr-o clasă, dacă variabilele de instanță ale clasei nu sunt în mod explicit inițializate (prin constructori), ele sunt în mod automat inițializate astfel:

int→0
byte→0
char→'\0' (cu caracterul al cărui cod Unicode este 0)
boolean→false
float→0.0
double→0.0

4. Cum se definesc constantele?

Constantele sunt variabile care au un conținut fix.

Constantele se declară în Java cu ajutorul cuvântului cheie **final**.

Exemplu:

final int NR_ELEVI=30;

~~NR_ELEVI=31;~~ eroare de compilare

Valoarea unei constante nu se modifică pe parcursul programului.

Se recomandă ca numele constantelor să fie scris cu litere mari.

5. Exemplu de clasă ce are numai date.

```
class NumaiDate
{
    public int x;
    public double y;
}
```


Compilarea:

javac NumaiDate.java ⇒ fişierul NumaiDate.class

Pentru a putea sa executam aplicația, trebuie sa scriem o clasă ce contine metoda main():

```
class Test
{
    public static void main(String args[ ])
    {
        // x←7; y←3.5;
        NumaiDate n=new NumaiDate( );
        n.x=7;
        n.y=3.5;
        System.out.println("x="+n.x);
        System.out.println("y="+n.y);
    }
}
```

6. Exemplu de clasă ce are numai metode.

Dăm ca exemplu o clasă ce are două metode: metoda main( ) şi metoda calculMedie( ).

```
import javax.swing.*;
class Medie
{
    public static void main(String args[ ])
    {
        int a=
            Integer.parseInt(JOptionPane.showInputDialog("a="));
        int b=
            Integer.parseInt(JOptionPane.showInputDialog("b="));
        int c=
            Integer.parseInt(JOptionPane.showInputDialog("c="));
        double m=calculMedie(a,b,c);
        System.out.println(m);
    }
```

OBS: În cazul aceleiaşi clase, dintr-o metodă statică nu se poate apela decât tot o metodă statică a aceleiaşi clase:

```
    private static double calculMedie(int x, int y, int z)
    {
```

```
        return (x+y+z)/3.0;
    }
}
```

7. Polimorfism parametric.

Prin **polimorfism** se înțelege: același nume – mai multe forme.
Polimorfismul este o noțiune specifică pentru POO.
Polimorfismul poate să apară la compilare sau la execuție.
Polimorfismul parametric este un polimorfism la compilare. Acesta se referă la faptul că într-o clasă putem avea mai multe metode ce au același nume, dar definiții (implementări) diferite. Metodele care au același nume diferă între ele prin lista de parametrii (și evident și prin implementare). Compilatorul știe încă din faza de compilare ce versiune de metodă să apeleze, pe baza numărului diferit de parametrii sau pe baza tipurilor diferite de parametrii. Acest mecanism se mai cheamă **overloading** (supraîncărcare).

Exemplu

Citim de la tastatură trei numere. Calculați cmmdc al primelor două numere și cmmdc pentru toate trei.

```
import javax.swing.*;
class CalculeCmmdc
{
    public static void main(String args[ ])
    {
        //citim trei numere întregi:
        int a=
        Integer.parseInt(JOptionPane.showInputDialog("a="));
        int b=
        Integer.parseInt(JOptionPane.showInputDialog("b="));
        int c=
        Integer.parseInt(JOptionPane.showInputDialog("c="));
        int rez1=cmmdc(a,b);
        System.out.println(rez1);
        int rez2=cmmdc(a,b,c);
        System.out.println(rez2);
    }

    private static int cmmdc(int a, int b)
```



```
    {
        while (a!=b)
            if (a>b) a=a-b;
            else b=b-a;
        return a;
    }
    private static int cmmdc(int a, int b, int c)
    {
        while (a!=b)
            if (a>b) a=a-b;
            else b=b-a;
        while (a!=c)
            if (a>c) a=a-c;
            else c=c-a;
        return a;
    }
}
```

8. Exemplu de clasă ce are și variabile de instanță și metode

Scriem clasa Cerc ce are ca variabile de instanță raza cercului și coordonatele lui, și ca metode:

```
setRaza( )
getRaza( );
setX0( );
setY0( )
getX0( )
getY0( );
arie( ).
```

```
class Cerc
{
    private int raza;
    private int x0,y0;
    public void setRaza(int raza)
    {
        this.raza=raza;
    }
    /* cuvântul cheie "this" este o referință către obiectul
       curent(obiectul instanțiat din clasa în care apare this).
```

```
    */
    public int getRaza( )
    {
        return raza;
    }
    public void setX0(int x)
    {
        x0=x;
    }
    public void setY0(int y)
    {
        y0=y;
    }

    public int getX0( )
    {
        return x0;
    }
    public int getY0( )
    {
        return y0;
    }
    public double arie()
    {
        return Math.PI*raza*raza;
    }
}
class TestCerc
{
    public static void main(String args[ ])
    {
        Cerc c=new Cerc( );
        c.setRaza(10);
        double a=c.arie( );
        System.out.println("arie="+a);
    }
}
```


9. Constructori.

Definiție: Constructorul este o metodă specială a unei clase care are același nume cu clasa, nu are în semnătură câmpul pentru tipul valorii returnate și este apelată în mod automat (implicit) la instanțierea unui obiect din clasa respectivă.

O clasă poate să aibă în mod explicit definiți zero, unul sau mai mulți constructori.

În mod tipic, un constructor face inițializările variabilelor de instanță.

Exemplul 1

Rescriem clasa Cerc în care definim explicit doi constructori: primul inițializează raza și coordonatele centrului cu (0,0) și al doilea inițializează raza și coordonatele centrului cu trei numere date ca parametrii.

```
class Cerc
{
    private int raza;
    private int x0,y0;
    // primul constructor:
    public Cerc(int r)
    {
        raza=r;
        x0=0;
        y0=0;
    }
    // al doilea constructor:
    public Cerc(int r, int x, int y)
    {
        raza=r;
        x0=x;
        y0=y;
    }
    public double arie( )
    {
        return Math.PI*raza*raza;
    }
}
```

Exemplu de utilizare pentru clasa Cerc:

```
class TestCerc
{
    public static void main(String args[ ])
    {
        Cerc c1=new Cerc(10,5,7);
        System.out.println("arie="+c1.arie( ));
        Cerc c2=new Cerc(3);
        System.out.println(c2.arie( ));
    }
}
```

Constructorii au fost introduşi în POO pentru a nu uita să facem iniţializările variabilelor de instanţă.

Exemplul 2

Scriem clasa Numar ce are ca variabilă de instanţă privată un număr întreg nr, şi ca metode:

- constructorul, ce iniţializează variabila nr;
- getNr(), ce returnează valoarea variabilei private nr;
- metoda estePatratPerfect() ce returnează *true* dacă nr este pătrat perfect şi *false* în caz contrar;
- metoda estePrim() ce returnează *true* dacă nr este prim şi *false* în caz contrar;
- metoda afisareDivizori() ce afişează divizorii numărului nr.

Scriem şi o clasă de test pentru clasa Numar.

```
class Numar
{
 private int nr;
 public Numar(int x)
 {
  nr=x;
 }
 public int getNr()
 {
        return nr;
 }
 public void afisareDivizori()
 {
  //nr. 1, este sigur divizor:
```



```
  System.out.println("1");
  //Cautam divizorii intre 2 si jumatatea numarului:
  for(int i=2;i<=nr/2;i++)
   if(nr%i==0)System.out.println(i);
  //numarul se divide cu el insusi:
  System.out.println(nr);
 }
 public boolean estePrim()
 {
  boolean este = true;
  for(int i=2;i<=Math.sqrt(nr);i++)
   if(nr%i==0){
      este = false;
      break;
     }
  return este;
  }
  public boolean estePatratPerfect()
  {
   int radical=(int)Math.sqrt(nr);
    if(radical*radical==nr)return true;
    else return false;
  }
 }

class TestNumar
{
 public static void main (String args[])
 {
  Numar n=new Numar(25);
  System.out.println("Divizorii numarului "+n.getNr());
  n.afisareDivizori();
  System.out.println("este patrat perfect="+n.estePatratPerfect());
  if(n.estePrim()==true)System.out.println(n.getNr()+" este prim.");
 else System.out.println(n.getNr()+" nu este prim.");
 }
}
```

Exemplul 3

Construim clasa Dreptunghi, ce are ca variabile de instanţă private, două numere întregi a şi b, ce reprezintă lungimile laturilor unui dreptunghi. În această clasă avem ca metode:

- constructorul, ce face iniţializările;
- metoda calculPerimetru(), ce returnează perimetrul dreptunghiului;
- metoda calculArie(), ce returnează aria dreptunghiului;
- metoda estePatrat(), ce returnează *true* dacă dreptunghiul este pătrat;
- metoda suntEgale(), ce are ca parametru un dreptunghi d şi scoate ca rezultat *true* dacă dreptunghiul curent (cel pentru care se apelează metoda) este egal cu dreptunghiul d.

Scriem şi o clasă de test pentru clasa Dreptunghi.

```
class Dreptunghi
{
 private int a;
 private int b;
 public Dreptunghi(int x,int y)
 {
  a=x;
  b=y;
 }
 public int calculPerimetru()
 {
  return 2*(a+b);
 }
 public int calculArie()
 {
  return a*b;
 }

 public boolean estePatrat()
 {
        if(a==b)return true;
        else return false;
 }

 public boolean suntEgale(Dreptunghi d)
 {
```



```
        if ((this.a==d.a)&&(this.b==d.b))return true;
        else return false;
 }
}
class TestDreptunghi
{
 public static void main (String args[])
 {
  Dreptunghi d=new Dreptunghi(5,7);
  System.out.println("Primetrul este "+d.calculPerimetru());
  System.out.println("Aria este "+d.calculArie());
  System.out.println("Dreptunghiul este patrat= "+d.estePatrat());
  Dreptunghi d1=new Dreptunghi(5,7);
  System.out.println("Sunt egale= "+d.suntEgale(d1));
 }
}
```

Ce trebuie adăugat în clasa Dreptunghi astfel încât pentru un obiect Dreptunghi să-i putem afişa lungimile laturilor? Vom prezenta trei soluţii.

Soluţia 1: Adăugăm metodele publice getX( ) şi getY( ) în clasa Dreptunghi.

```
public int getX( )
{
  return x;
}

public int getY( )
{
  return y;
}
```

În main( ):

```
System.out.println("x="+d.getX( ));
System.out.println("y="+d.geYX( ));
```

Soluţia 2: Scriem metoda publică afişare() în clasa Dreptunghi.

```
public void afişare( )
{
   System.out.println("x="+x);
```

```
  System.out.println("y="+y);
}
```

În main( ):

```
d.afişare( );
```

Soluţia 3: (nerecomandată) Facem variabilele x şi y publice în clasa Dreptunghi şi le accesăm direct din main() cu: *d.x* şi *d.y*.

Exemplul 4

Dezvoltam clasa DouaNumere, ce are variabile de instanţă două numere întregi a şi b, şi ca metode:

- constructorul ce face iniţializările;
- metoda maxim(), ce returnează maximul dintre a şi b;
- metoda cmmdc(), ce returnează cel mai mare divizor comun al celor două numere.

Scriem şi o clasă de test pentru clasa DouaNumere.

```
class DouaNumere
{
 private int a,b;
 public DouaNumere(int n1, int n2)
 {
  a=n1;
  b=n2;
 }
 public int maxim()
 {
       if(a>b)return a;
       else return b;
 }
 public int cmmdc()
 {
  int A=a; int B=b;
  while(A!=B)
   if(A>B)A=A-B;
   else B=B-A;
  return A;
 }
}
class Test2Numere
 {
```



```
 public static void main(String args[])
 {
  DouaNumere d=new DouaNumere(12,18);
  System.out.println("maximul este: "+d.maxim());
  System.out.println("cmmdc este: "+d.cmmdc());
 }
}
```

Exemplul 5

Scriem clasa TreiNumere, ce are variabile de instanţă trei numere întregi a,b şi c, şi ca metode:

- constructorul ce face iniţializările;
- metoda maxim(), ce returnează maximul dintre a, b şi c;
- metoda suntPitagorice(), ce returnează *true*, dacă a, b, c sunt numere pitagorice.

Scriem şi o clasă de test pentru clasa TreiNumere.

```
class TreiNumere
{
 private int a,b,c;
 public TreiNumere(int n1, int n2, int n3)
 {
  a=n1;
  b=n2;
  c=n3;
 }
 public int maxim()
 {
       int max=a;
       if(b>max)max=b;
       if(c>max)max=c;
       return max;
 }
 public boolean suntPitagorice()
 {
 if((a*a==b*b+c*c)||(b*b==a*a+c*c)||(c*c==a*a+b*b))
   return(true);
 else return(false);
 }
}
class Test3Numere
```

```
{
 public static void main(String args[])
 {
  TreiNumere t=new TreiNumere(3,5,4);
  System.out.println("maximul este: "+t.maxim());
  if(t.suntPitagorice())System.out.println("Sunt numere pitagorice");
  else System.out.println("Nu sunt numere pitagorice");
 }
}
```

Exemplul 6

Să se dezvolte clasa Complex, ce are variabile de instanță private două numere întregi re şi im (partea reală şi partea imaginară a unui număr complex) şi ca metode:

- constructorul ce face inițializările;
- modul(), ce returnează modulul numărului complex;
- suma(), ce are ca parametru un număr complex c, prin care la numărul complex curent se adună numărul complex c (rezultatul se depune în numărul curent);
- produs(), ce are ca parametru un număr complex c, prin care în numărul complex curent se depune rezultatul înmulțirii dintre numărul complex curent şi numărul complex c;
- getRe(), ce returnează partea reală a numărului complex;
- getIm(), ce returnează partea imaginară a numărului complex;
- equals(), ce redefineşte metoda equals() din clasa Object, prin care se compară din punct de vedere al conținutului, două obiecte Complex: obiectul curent şi obiectul dat ca parametru;
- toString(), ce redefineşte metoda toString() din clasa Object, prin care se dă o reprezentare sub formă de String a unui număr complex;

Scriem şi o clasa de test pentru clasa Complex.

```
class Complex
{
 private double re;
 private double im;
 public Complex(double x, double y)
 {
  re=x;
  im=y;
 }
```



```
 public double getRe()
 {
  return re;
 }
 public double getIm()
 {
  return im;
 }
 public double modul()
 {
  return Math.sqrt(re*re+im*im);
 }
 //adunarea nr. complex curent, cu un alt nr. complex, cu depunerea
 //rezultatului in numarul complex curent:
 public void suma(Complex c)
 {
  re=re+c.re;
  im=im+c.im;
 }
 //inmultirea nr. complex curent, cu un alt nr. complex, cu depunerea
 //rezultatului in numarul complex curent:
 public void produs(Complex c)
 {
  re=re*c.re-im*c.im;
  im=re*c.im+im*c.re;
 }

 //redefinirea metodei equals() din clasa parinte Object:
 //(trebuie sa se pastreze aceeasi semnatura:)
 public boolean equals(Object obj)
 {
   Complex c=(Complex)obj;
    if(c!=null)
    if((re==c.re)&&(im==c.im))return true;
   return false;
 }
 //redefinirea metodei toString() din clasa parinte Object:
 public String toString()
 {
  String s="("+re+","+im+")";
```

```
 return s;
}
}

class TestComplex
{
public static void main(String args[])
{
 Complex c1=new Complex(1,1);
 System.out.println("Modulul este= "+c1.modul());
 Complex c2=new Complex(1,1);
 c1.suma(c2);
 System.out.println("suma = "+c1.toString());
 Complex c3=new Complex(1,1);
 System.out.println("sunt egale: "+c2.equals(c3));
}
}
```

Exemplul 7

Scriem clasa Unghi, ce are ca variabilă de instanță privată un număr întreg x, măsura în grade a unui unghi, şi ca metode:

- constructorul;
- suntComplementare(), ce are ca parametru un alt unghi u, şi care returnează *true* dacă unghiul u este complementar cu unghiul curent;
- conversieRadiani(), ce returnează valoarea exprimată în radiani a unghiului curent x.

Scriem şi o clasă de test pentru clasa Unghi.

```
class Unghi
{
 private int x;
 public Unghi(int x)
 {
  this.x=x;
 }
 public boolean suntComplementare(Unghi u)
 {
  if(this.x+u.x==90) return true;
  else return false;
 }
```



```
 public double conversieRadiani()
 {
  return (Math.PI*x)/180;
 }

}
class TestUnghi
{
 public static void main (String args[])
 {
  Unghi a=new Unghi(30);
  System.out.println("Radiani= "+a.conversieRadiani());
  System.out.println("sunt  complementare="+
                     a.suntComplementare(new Unghi(60)));
 }
}
```

Exemplul 8

Să se dezvolte clasa Timp ce are ca variabile de instanță trei numere întregi: h, min, sec (ore, minute, secunde). Ca metode:

- constructorul;
- conversieInSecunde() ce returnează timpul curent, exprimat în secunde;
- compara(), ce are ca parametru un Timp t, şi care returnează 1 dacă timpul current este mai mare ca t, 0 dacă cei doi timpi sunt egali , şi -1 în caz contrar.

Scriem şi o clasă de test.

```
class Timp
{
 private int h; //ore
 private int min;
 private int sec;
public Timp(int ore,int m, int s)
{
 h=ore;
 min=m;
 sec=s;
}
```

```
 return 3600*h+60*min+sec;
}
public int compara(Timp t)
//returneaza 0 daca cei doi timpi sunt egali
//returneaza 1 daca timpul curent > timpul t dat ca argument
//returneaza -1 daca timpul curent < timpul t
{
 int sec1=this.conversieSecunde();//timpul curent, in secunde
 int sec=t.conversieSecunde();//timpul t, in secunde
 if(sec1>sec)return 1;
 else if(sec1==sec)return 0;
 else return -1;
}
}

class TestTimp
{
public static void main(String args[])
{
Timp t1=new Timp(1,5,6);
Timp t2=new Timp(1,5,6);
int rezultat=t1.compara(t2);
if(rezultat==1)System.out.println("t1>t2");
else if(rezultat==0)System.out.println("t1=t2");
else System.out.println("t1<t2");
}
}
```


Capitolul 4
Vectori. Matrici.

1. **Vectori intrinseci.**
2. **Exemplu de metode ce au ca parametrii vectori intrinseci.**
3. **Matrici.**

1. Vectori intrinseci

Vectorii intrinseci sunt tablouri unidimensionale.

Spre deosebire de limbajul C, în limbajul Java, vectorii intrinseci sunt obiecte, instanțe ale unei clase intrinsecă limbajului.

Fiind obiecte, vectorii intrinseci (pe scurt ii vom numi vectori) se instanțiază cu ajutorul operatorului **new**.

Numele vectorului, ca și în limbajul C, este o referință către zona de memorie unde se memorează obiectul vector.

Componentele vectorului se accesează prin indexare, ca și în C.

Un vector, cu ajutorul operatorului **new,** se instanțiază conform sintaxei:

tip numeVector[ ]=new tip[dimensiune];

Exemplu: int a[ ]=new int[10];
Componentele vectorului sunt:
a[0], a[1], ... ,a[9]

Un vector poate fi creat si prin listarea directa a valorilor lui initiale.

Exemplu:

```
int b[]={1,5,-1};
```

Aceasta instructiune, putea fi realizata echivalent, astfel:

```
int b[]=new int[3];
b[0]=1;
b[1]=5;
b[2]=-1;
```

OBS: **1.** Spre deosebire de limbajul C, în Java, se verifică indexarea în afara granițelor și dacă se face această indexare, avem eroare la execuție.

2. Spre deosebire de limbajul C, dimensiunea vectorului poate să fie și o variabilă, ce a fost inițializată.

Exemplu:

C: ~~int dim=10;~~ **Java**: int dim=10;

~~int a[dim];~~ int a[ ]=new int[dim]; //da!

3. Atenție la diferența între declararea unui vector şi definirea sau crearea sau instanțierea lui.

Exemplu:

double b[ ]; // declarare vector

b=new double [10]; // Crearea obiectului vector:

Vectorii pot contine ca elemente ,nu numai date primitive, ci si obiecte. Exemplu:

Integer v1[]=new Integer[3];

Sau:

Integer v2[]={new Integer(1), new Integer(3), new Integer(4)};

În clasa din care se instanțiază vectorii, este definită variabila de instanță publică **length** de tip read-only (poate fi doar citită, nu şi modificată) în care, în mod automat, la instanțierea vectorului, se memorează dimensiunea vectorului.

Pentru a modela vectori a căror dimensiune poate fi modificată, **vectori dinamici**, se foloseşte **clasa Vector** ce este definită în **java.util**.

Accesarea componentelor unui vector din clasa Vector nu se mai face prin indexare, ci prin metode: **get( ), add( ),** etc..

Exemplul 1

Citim un număr natural N. Citim N numere întregi într-un vector a. Să calculăm şi afişăm maximul din vector.

```
class CalculMaxim
{
 public static void main(String args[ ])
 {
  int N=Integer.parseInt(JOptionPane.showInputDialog("N="));
  int a[ ]=new int[N];
  int i;
  for (i=0; i<N; i++)
   a[i]= Integer.parseInt(JOptionPane.showInputDialog("nr="));
  int max=a[0];
  for (i=1; i<N; i++)
   if (a[i]>max) max=a[i];
    System.out.println(max);
```



```
 }
}
```

Exemplul 2

Se citeşte de la tastatură un număr natural N; se instanțiază un vector de N numere întregi. Să se completeze acest vector cu numere aleatoare în gama 0..N-1, cu condiția ca fiecare număr din această gamă să apară o singură dată.

Algoritm: vom inițializa vectorul cu numerele 0,1, .., N-1, date în această ordine. Apoi, aceste numere inițiale, le vom comuta, pozițiile de comutare fiind generate aleator.

```
import javax.swing.*;
import java.util.*;
class InitNumAleatoare
{
 public static void main(String args[])
 {
  int N;
 String s = JOptionPane.showInputDialog("N=");
 N = Integer.parseInt(s);
  int a[] = new int [N];
  int i;
 //se initializeaza vectorul cu numerele 0,1,...,N-1, in aceasta ordine:
  for(i=0;i<a.length;i++)
  a[i] = i;
 Random r = new Random();
 //se repeta de N ori:
  for(i=0;i<N;i++){
    //se genereaza doua numere aleatoare n1 si n2:
    int n1 = r.nextInt(N);
    int n2 = r.nextInt(N);
    //se comuta variabilele a[n1] si a[n2]:
    int aux = a[n1];
    a[n1] = a[n2];
    a[n2] = aux;
   }
 //Afisare vector generat:
  for (i=0;i<N;i++)
 System.out.println(a[i]);
 System.exit(0);
```

```
}
}
```

Exemplul 3

Se citeşte de la tastatură un număr natural N; se instanţiază un vector de N numere întregi. Să se completeze acest vector cu numere aleatoare în gama 0..N-1, cu condiţia ca fiecare număr din această gamă să apară o singură dată. (Problema anterioară, dar un alt algoritm: vom introduce pe rând în vector, câte un număr aleator, cu condiţia ca să fie diferit de numerele anterioare introduse).

```
import java.util.*;
class Aleatoare
{
        public static void main(String args[])
        {
           final int N=10;
           int a[]=new int[N];
           Random r=new Random();
           int i;
           int nr;
           for(i=0;i<N;i++){
              for(;;){
               nr=r.nextInt(N);//intre 0 si (N-1)
              //a mai fost generat?
              //cautare liniara:
              boolean estePrezent=false;
              for(int j=0;j<i;j++)
                if(nr==a[j]){
                   estePrezent=true;
                   break;}
              if(estePrezent==false){
                a[i]=nr;
                break;
               }
              }//for;;
            }//for i
            //afisare:
            for(i=0;i<N;i++)
              System.out.print(a[i]+" ");
            System.out.println("\n");
```



```
        }//main
}
```

Exemplul 4

Se citesc de la tastatură doi vectori a şi b, ce au aceeaşi dimensiune (dimensiunea lor comună, N, este cunoscută). Să se afişeze dacă vectorul b este o permutare a vectorului a.

```
import javax.swing.*;
import java.util.*;
class SuntPermutari
{
 public static void main(String args[])
 {
  int N;
 String s = JOptionPane.showInputDialog("N=");
 N = Integer.parseInt(s);
 int a[] = new int [N];
 int b[] = new int [N];
 int i;
for(i=0;i<N;i++)
 a[i]=Integer.parseInt(JOptionPane.showInputDialog("a["+i+"]="));
for(i=0;i<N;i++)
 b[i]=Integer.parseInt(JOptionPane.showInputDialog("b["+i+"]="));
//sortam cei doi vectori:
Arrays.sort(a);
Arrays.sort(b);
//comparam cei doi vectori:
boolean suntPerm=true;
for(i=0;i<N;i++)
        if(a[i]!=b[i]){
                suntPerm=false;
                break;
        }
if(suntPerm)System.out.println("sunt");
else System.out.println("nu sunt");
}
}
```

2. Exemplu de metode ce au ca parametrii vectori intrinseci.

Spre deosebire de limbajul C, unde într-o funcţie ce avea ca parametru un vector, trebuia dat ca parametru şi dimensiunea vectorului, în Java, dimensiunea vectorului nu mai este necesară ca parametru, deoarece dimensiunea vectorului este memorată în variabila publică length.

Exemplul 1

Citim de la tastatură doi vectori a şi b ce au aceeaşi dimensiune N. Să afişăm dacă:

a) vectorul a are toate elementele diferite între ele.
b) vectorul a este egal cu vectorul b.

Vom folosi două metode separate: metoda suntDiferite( ) şi metoda compară( ).

```
class Compara
{
 public static void main(String args[ ])
 {
 //dimensiunea comună:
 int N=Integer.parseInt(JOptionPane.showInputDialog("nA="));
 int i;
 int a[ ]=new int[N];
 // citire a[ ]:
 for (i=0; i<N; i++)
  a[i]= Integer.parseInt(JOptionPane.showInputDialog("nr_a="));
 int b[ ]=new int[N];
 // citire b[ ]:
 for (i=0; i<N; i++)
  b[i]= Integer.parseInt(JOptionPane.showInputDialog("nr_b="));
 boolean sunt=suntDiferite(a);
 System.out.println("sunt diferite : "+sunt);
 boolean egali=compară(a,b);
 System.out.println("a[ ]=b[ ]:"+egali);
 }
 private static boolean suntDiferite(int a[])
 {
  int i,j;
  for(i=0;i<a.length-1;i++)
   for(j=i+1;j<a.length;j++)
```



```
   if(a[i]==a[j])return false;
  return true;
 }

 private static boolean compară(int a[ ], int b[ ])

 int i;
 for (i=0; i<a.length; i++)
  if (a[i]!=b[i]) return false;
 return true;
 }
```

Exemplul 2

În clasa System, care face parte din pachetul java.lang, este definită metoda: currentTimeMillis() cu semnătura:
public static long int currentTimeMillis().
Această metodă returnează valoarea timpului sistemului, în milisecunde.
Se generează N= 100000 numere întregi aleatoare, în gama 0...9999, care se memorează în vectorul a. Folosind metoda currentTimeMillis(), să se calculeze durata sortării în ordine crescătoare a vectorului a. Se va sorta vectorul mai întâi cu metoda Arrays.sort(), şi apoi acelaşi vector, cu metoda de sortare prin interschimbare.

```
import javax.swing.*;
import java.util.*;
class DurataSortare
{
 public static void main(String args[])
 {
  final int N=100000;
  final int GAMA=1000;
  int a[] = new int [N];
  Random r=new Random();
  int i;
  for(i=0;i<N;i++)
  a[i]=r.nextInt(GAMA);
  //salvam vectorul a:
  int b[] = new int [N];
```

```
 for(i=0;i<N;i++)
 b[i]=a[i];
 long t1=System.currentTimeMillis();
 Arrays.sort(a);
 long t2=System.currentTimeMillis();
 System.out.println(t2-t1);
 //Acelasi vector sortat prin interschimbare:
 long t3=System.currentTimeMillis();
 sortare(b);
 long t4=System.currentTimeMillis();
 System.out.println("sortare interschimbare : "+(t4-t3));
}
private static void sortare(int a[])
{
 for(int i=0;i<a.length-1;i++)
 for(int j=i+1;j<a.length;j++)
 if(a[i]>a[j]){
    int aux=a[i];
    a[i]=a[j];
   a[j]=aux;
  }
 }
}
```

Exemplul 3

Să se scrie o aplicație ce folosește o metodă ce are doi parametrii: un vector de numere întregi a , și un parametru de tip caracter denumit sens, ce poate avea două valori: 's' (stânga) și 'd' (dreapta). În metodă se rotesc în vectorul a toate elementele, cu o poziție, spre stânga sau spre dreapta, după valoarea parametrului sens.
Exemplu:
a={1,2,30} şi sens='d'. După execuția metodei avem: a={30, 1, 2}

```
class Rotire
{
 public static void main(String args[])
 {
  int a[]={1,2,30};
  rotire(a,'d');
  for(int i=0;i<a.length;i++)
  System.out.print(a[i]+" ");
```



```
 }
 private static void rotire(int a[],char sens)
 {
  int N=a.length;//dimensiune vector a[]
  if(sens=='s'){
   int aux=a[0];
   for(int i=0;i<N-1;i++)
                a[i]=a[i+1];
               a[N-1]=aux;
  }
  else if(sens=='d'){
              int aux=a[N-1];
              for(int i=N-2;i>=0;i--)
               a[i+1]=a[i];
               a[0]=aux;
  }
 }
}
```

Exemplul 4

Scrieți o aplicație în care se implementează algoritmul de căutare binară a prezenței unui număr x citit de la tastatură într-un vector sortat crescător.

```
class CautareBinara
{
public static void main(String args[])
{
 int a[]={1,5,7,9,12,30};
 int x=1;
 //Este prezent x in vectorul a[] ?
 if(estePrezent(x,a,0,a.length-1))System.out.println("Este prezent.");
 else System.out.println("Nu este prezent.");
}

private static boolean estePrezent(int x,int a[],int s,int d)
{
 if(s>d) return false;
 if(s==d){
  if(x==a[s])return true;else return false;}
 int m;
```

```
 m=(s+d)/2;
 if(x==a[m])return true;
 else if(x<a[m])return estePrezent(x,a,s,m-1);
 else return estePrezent(x,a,m+1,d);
}//estePrezent
}
```

Exemplul 5

Scrieți o aplicație Java în care se implementează algoritmul de sortare prin metoda selecției maximului.

```
import javax.swing.*;
class SelectieMaxim{
 public static void main(String args[]){
  final int N=5;
  int A[]=new int[N];
  int i;
  //citire vector  A[]:
  for(i=0;i<N;i++)
  A[i]=Integer.parseInt(JOptionPane.showInputDialog("nr="));
  ordonareCresc(A,N);
  System.out.println("Numerele sortate crescator: ");
  for(i=0;i<N;i++)
    System.out.println(A[i]);
 }//main
 private static void ordonareCresc(int A[], int nA)
 //trebuie oblig. ca parametru sa fie dat si nA, pt. ca metoda se va
  // apela recursiv pt. mai putine elemente.
 {
   if(nA==1)return;//un singur element, este implicit ordonat!
   int poz=getPozMax(A,nA);//indexul maximului
   if(poz!=nA-1){//daca este chiar pe ultima poz., nu avem ce comuta!
   //comut elementul de pe utima poz. din A cu maximul gasit:
    int temp=A[nA-1];
    A[nA-1]=A[poz];
    A[poz]=temp;}
    //ordoneaza recursiv portiunea din vectorul A[]
    //  ce are doar nA-1 componente:
    ordonareCresc(A,nA-1);
 }//ordonareCresc
  private static int getPozMax(int A[], int nA)
```



```
 {
  int max=A[0];
  int pozMax=0;
  for(int i=1;i<nA;i++)
    if(A[i]>max){
          max=A[i];
          pozMax=i;}
  return pozMax;
 }
}
```

Exemplul 6

Folosind clasa Vector, implementați următorul algoritm:

se repetă
citim un număr real
îl memorăm într-un obiect de tip Vector
se afişează mesajul: "Dati un alt numar? [d/n] "
se citeşte răspunsul la acest mesaj
până când răspunsul este 'n'

Se vor afişa în final numerele memorate în obiectul Vector

```
import java.util.*;
import javax.swing.*;
class VectorDinamic
{
 public static void main(String args[])
 {
  Vector v=new Vector();
  double nr;
  for(;;){
   nr=Double.parseDouble(JOptionPane.showInputDialog("nr.="));
   v.add(new Double(nr));
   String raspuns=
   JOptionPane.showInputDialog("Dati un alt    numar? [d/n]: ");
   if(raspuns.charAt(0)=='n')break;
   }
  afisareVector(v);
  //Modificam primul numar introdus anterior in Vector:
  v.set(0,new Double(7.5));
  System.out.println("Vectorul modificat: ");
```

```
  afisareVector(v);
  System.out.println();
 }
 private static void afisareVector(Vector v)
 {
  int n=v.size();// dimensiunea obiectului Vector:
  for(int i=0;i<n;i++){
    Double obiectCrt=(Double)v.get(i);
    //fara operatorul cast (Double): eroare, caci metoda get()
    //returneaza un obiect de tipul Object
    System.out.println(obiectCrt.doubleValue());
    //dintr-un obiect Double se extrage valoarea lui numerica de tipul
    //double, cu metoda doubleValue()
  }//for
 }
}
```

Exemplul 7

Să se scrie o aplicație Java în care se implementează algoritmul de sortare bubblesort a unui vector de numere întregi.

```
class Sortare
{
 public static void main(String args[])
 {
  int a[]={1,7,6,5,10,4,3,2,9,8};
  int i;
  bubbleSort(a);
  //afisare:
  for(i=0;i<a.length;i++)
   System.out.println(a[i]);
 }
 private static void bubbleSort(int a[])
 {
  int dim=a.length;
  int i;
  boolean suntInv;
  for(;;){
   suntInv=false;
   for(i=0;i<dim-1;i++)
    if(a[i]>a[i+1]){
```



```
     int aux=a[i];
     a[i]=a[i+1];
     a[i+1]=aux;
     suntInv=true;
    }
   if(suntInv==false)break;
  }
 }
}
```

Exemplul 8

Se citesc două mulțimi de numere întregi, (în fiecare mulțime, numerele sunt diferite), în doi vectori A şi B. Să se calculeze reuniunea celor două mulțimi.

```
class Reuniune
{
 public static void main(String args[]){
  int A[]={1,10,2};
  int B[]={5, 6, 7, 2, 10};
  int nA=A.length;//nr. de elemente din A
  int nB=B.length;
  int i;
  int C[]=new int[nA+nB];//vectorul reuniune
  //Copiem pe A in C:
  for(i=0;i<nA;i++)
    C[i]=A[i];
  //Adaugam in C[] toate elementele din B[], care
  // nu sunt si in A[]:
  int iC=nA;//index in multimea C
  for(i=0;i<nB;i++)
    if(estePrezent(B[i],A)==false){
          C[iC]=B[i];
          iC++;
    }
  int nC=iC;//numarul de elemente din C[]
  //Afisare A[]:
  for(i=0;i<nA;i++)
   System.out.print(A[i]+" ");
   System.out.println();
  //Afisarea multimii B:
```

```
 for(i=0;i<nB;i++)
  System.out.print(B[i]+" ");
  System.out.println();
//Afisarea multimii C:
 for(i=0;i<nC;i++)
  System.out.print(C[i]+" ");
}
  private static boolean estePrezent(int x,int a[])
{
  for(int i=0;i<a.length;i++)
    if(a[i]==x)return true;
      return false;
  }
}
```

3. Matrici

Matricile reprezintă un caz particular de vectori. O matrice este un vector ale cărui componente sunt tot vectori (liniile matricii).

O matrice se instanțiază astfel:

tipDate nume[ ][ ]=new tip[nrLinii][nrColoane];

Exemplu: int a[ ][ ]=new int[10][20];

O matrice poate fi inițializată și în mod direct, ca în exemplul următor:

Daca a=$\begin{pmatrix} -3 & 0 \\ 1 & 2 \\ -5 & 7 \end{pmatrix}$, o putem initializa direct, astfel:

```
int a[ ][ ]={{-3,0},
             {1,2},
             {-5,7}};
```

O matrice fiind un vector de vectori (de linii), plecând de la numele matricii, obținem numărul de linii și numărul de coloane, astfel:

nrLinii=nume.length

nrColoane=nume[0].length

Exemplul 1

Citim de la tastatură numărul de linii și numărul de coloane ale unei matrici. Inițializăm matricea cu numere aleatoare in gama 0...99 și calculăm și afișăm maximul din matrice.

```
class M
{
 public static void main(String args[ ])
 {
  int nL, nC;
  nL=Integer.parseInt(JOptionPane.showInputDialog("nr.linii="));
  nC=Integer.parseInt(JOptionPane.showInputDialog("nr.col.="));
  int a[ ][ ]=new int[nL][nC];
  final int GAMA=100;
  Random r=new Random( );
  for(int i=0;i<nL;i++)
   for(int j=0;j<nC;j++)
    a[i][j]=r.nextInt(GAMA);
 System.out.println("max="+maxim(a));
 }
 private static int maxim(int a[ ][ ])
 {
  int max=a[0][0];
  for(int i=0;i<a.length;i++)
   for(int j=0;j<a[0].length;j++)
    if(a[i][j]>max) max=a[i][j];
  return max;
 }
 }
```

Exemplul 2

Se citește de la tastatură o matrice de numere întregi, de dimensiuni cunoscute (numărul de linii și numărul de coloane citite anterior). Să se calculeze și afișeze dacă matricea are toate numerele egale între ele.

```
import javax.swing.*;
class MatriceSuntEgale
{
 public static void main(String args[])
 {
  int nL;//numar linii matrice
```

```
  nL=Integer.parseInt(JOptionPane.showInputDialog("nr. linii="));
  int nC;//numar coloane matrice
  nC=Integer.parseInt(
                    JOptionPane.showInputDialog("nr. coloane="));
  int a[][]=new int[nL][nC];
  //citire matrice:
  int i,j;
  for(i=0;i<nL;i++)
   for(j=0;j<nC;j++)
    a[i][j]=Integer.parseInt(JOptionPane.showInputDialog("nr="));
  boolean egale=suntToateEgale(a);
  if(egale==true)System.out.println("Sunt toate numerele egale.");
  else System.out.println("Nu sunt toate egale");
 }
 private static boolean suntToateEgale(int a[][])
 {
  for(int i=0;i<a.length;i++)
    for(int j=0;j<a[0].length;j++)
      if(a[i][j]!=a[0][0])return false;
  return true;
 }
}
```

Exemplul 3

Se dă o matrice de numere întregi. Să se calculeze şi afişeze dacă în acestă matrice există cel puţin două linii egale între ele.

```
class MatriceLiniiEgale
{
public static void main(String args[])
{
 int a[][]={{1,2,3,4,5},
            {0,1,0,0,0},
            {1,2,3,4,5},
            {1,1,1,1,1}};
//Comparam liniile matricii, doua cate doua:
 int nL=a.length;//numarul de linii
 int nC=a[0].length;//numarul de coloane
 int i,j;
 for(i=0;i<nL-1;i++)
  for(j=i+1;j<nL;j++)
```



```
    //compara linia i cu linia j:
     if(suntEgale(a[i],a[j])){
      System.out.println("Are linii egale intre ele");
      return;
      }
 System.out.println("Nu are linii egale intre ele")    ;
}
private static boolean suntEgale(int linie1[], int linie2[])
{
 //Se compara doi vectori pentru egalitatea lor:
 for(int i=0;i<linie1.length;i++)
   if(linie1[i]!=linie2[i])return false;
 return true;
}
}
```

Exemplul 4

Se dă o matrice de numere întregi.. Să se calculeze şi afişeze dacă în acestă matrice există cel puţin o linie care să aibă toate elementele egale între ele.

```
class MatriceLinieConstanta
{
 public static void main(String args[])
 {
  int a[][]={{1,2,3,4,5},
             {0,0,0,0,0},
             {1,2,3,4,5},
             {1,1,1,0,1}};
   //Parcurgem toate liniile matricii:
   int nL=a.length;//numarul de linii
   for(int i=0;i<nL;i++)
    //are linia curenta i, toate elementele egale?
    if(areToateEgale(a[i])){
     System.out.println("Are.");
     return;
    }
 System.out.println("Nu are.");
}
 private static boolean areToateEgale(int linie[])
 {
```

```
//Se compara toate elementele vectorului, cu primul:
for(int i=1;i<linie.length;i++)
 if(linie[i]!=linie[0])return false;
 return true;
 }
}
```


Capitolul 5
Stringuri

1. Clasa String. Generalităţi.
2. Metode de bază din clasa String.
3. Metode ce au ca parametrii stringuri.
4. Alte operaţii cu stringuri.
5. Vectori de şiruri.
6. Clasa StringTokenizer.

1. Clasa String. Generalitati.

În limbajul C, un string este un vector de caractere.

În limbajul Java, un string este un obiect instanţiat din clasa **String**, clasă definită în **java.lang**.

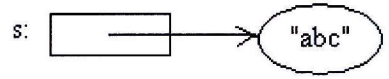
Instanţierea unui string se face în două moduri:

a) cu ajutorul operatorului **new**:

String s=new String("abc");

Variabila s este o variabilă referinţă ce ţine adresa obiectului string propriu-zis.

Obiectul string este alocat în memoria heap.



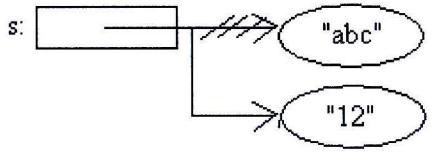
b) Al doilea mod de instanţiere: prin **atribuire directă.**

String s1="1234";

Stringurile sunt obiecte **imutabile** (nu se pot modifica).

Exemplu:

```
String s="abc";
.....
s="12";
```



În limbajul C++, există noţiunea de **destructor** ca şi noţiune complementară constructorului. Un destructor în limbajul C++ este apelat în mod explicit pentru a elibera, în general, memoria dinamică atunci când un obiect nu mai este folosit.

În limbajul Java, nu există noțiunea de destructor, eliberarea spațiului de memorie dinamică atunci când un obiect nu mai este necesar, se face automat de către un proces (fir de execuție) denumit **garbage collector** (colectorul de gunoi). Acest garbage collector sesizează când un obiect nu mai este necesar şi dezalocă în mod automat spațiul de memorie ocupat de acesta.

În cazul stringurilor asupra cărora trebuie făcute multe modificări, se recomandă să se folosească în loc de clasa String, clasa **StringBuffer** din pachetul **java.util**, stringurile instanțiate din această clasă fiind **mutabile** (pot fi modificate).

2. Metode de baza din clasa String.

1. Cea mai importantă metodă este **metoda length( )** cu semnătura:
public int length( )
care returnează lungimea şirului.
2. metoda charAt( ) cu semnătura:
public char charAt(int index)
care returnează caracterul de la poziția index.

ATENȚIE: in limbajul **C:** s[i]
In limbajul **Java:** s.charAt(i)
OBS: Pentru a citi şirul de la tastatură se foloseşte metoda **showInputDialog( )** din **clasa JOptionPane** care face parte din pachetul **javax.swing**.

Exemplu
Citim un şir de la tastatură. Să afişăm de câte ori apare caracterul a în şir.(pentru "casa" ⇒ 2).

```
import javax.swing.*;
class S1
{
 public static void main(String args[ ])
 {
  String s=JOptionPane.showInputDialog("şir=");
  int contor=0;
  int i;
  for (i=0; i<s.length( ); i++)
   if (s.charAt(i)= ='a') contor ++;
  System.out.println(contor);
```



```
 }
}
```

3. Metode ce au ca parametrii stringuri.

Exemplul 1
Rescriem programul anterior folosind o metodă separată ce returnează numărul de caractere egale cu un caracter dat ca parametru dintr-un string.

```
import javax.swing.*;
class S2
{
 public static void main(String args[ ])
 {
  String s=JOptionPane.showInputDialog("şir=");
  int contor=calcul(s,'a');
  System.out.println(contor);
 }
 private static int calcul(String s, char ch)
 {
  int contor=0;
  for (int i=0; i<s.length( ); i++)
   if (s.charAt(i)= ='a') contor ++;
  return contor;
 }
}
```

Exemplul 2
Să scriem o metodă ce are ca parametru un string şi care returnează adevărat dacă toate caracterele din şir sunt egale între ele.

```
private static boolean toateEgale(String s)
{
 for (int i=1; i<s.length( ); i++)
   if (s.charAt(i)!=s.charAt(0)) return false;
 return true;
}
```

Exemplul 3

Să scriem o metodă ce stabileşte dacă toate caracterele unui şir sunt diferite între ele.

```
private static boolean toateDiferite(String s)
{
 for (int i=0; i<s.length()-1; i++)
  for (int j=i+1; j<s.length(); j++)
   if (s.charAt(i)= =s.charAt(j)) return false;
 return true;
}
```

4. Alte operaţii cu stringuri.

Concatenarea a două şiruri:

Pentru a concatena două şiruri se foloseşte operatorul de concatenare **"+"**.

Exemplu:

```
String s;
String s1="abc";
String s2="12";
s=s1+s2; //s←"abc12".
```

Operatorul de concatenare "+" este un operator binar în care unul dintre operanzi trebuie să fie în mod obligatoriu String, cel de-al doilea poate să fie şi diferit de String, dar va fi convertit în mod automat, dacă este posibil, la tipul String.

Exemplu:

```
String s="abc";
int nr=15;
String rezultat=s+nr; // "abc15".
```

Compararea a două şiruri.

Sunt trei posibiltăţi de a compara două şiruri:

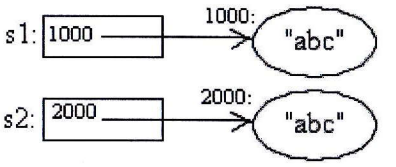
a) cu ajutorul operatorului = =
b) cu ajutorul metodei **equals( )**
c) cu ajutorul metodei **compareTo( )**

a) Compararea a două şiruri cu ajutorul operatorului = =

Se compara referintele, nu continutul !

Exemplu:

```
String s1="abc";
String s2= new String("abc");
if (s1= =s2) System.out.println("DA");
else System.out.println("NU");//Programul afişează: NU
```



Pentru a compara două stringuri ca şi conţinut se foloseşte metoda **equals( )** sau metoda **compareTo( )**.

b) Compararea a două şiruri cu ajutorul metodei equals( )

Metoda **equals( )** este definită în clasa Object şi este moştenită ca atare şi de către clasa String. Metoda equals( ) are următoarea semnătură:

public boolean equals(Object o)

Compară obiectul curent cu obiectul dat ca parametru. Dacă sunt egale returnează true.

Exemplu de aplicare pentru clasa String:

```
String s1="abc";
String s2= new String("abc");
if (s1.equals(s2)= =true) System.out.println("DA");
else System.out.println("NU");//Programul afişează DA
```

c) Compararea a două şiruri cu ajutorul metodei compareTo( )

Metoda **compareTo( )** este definită în clasa String şi are următoarea semnătură:

public int compareTo(String s)

Compară şirul curent cu şirul dat ca parametru din punct de vedere al ordinii lexicografice. Ordinea lexicografică este o generalizare a ordinii alfabetice, în care două şiruri sunt comparate caracter cu caracter, pe baza codurilor **Unicode** ale caracterelor.

Metoda **compareTo( )** scoate un rezultat 0, dacă cele două şiruri sunt egale(au acelaşi conţinut). Dacă şirul curent este mai mare decât şirul dat ca parametru, în sensul ordinii lexicografice, metoda compareTo( ) scoate un rezultat mai mare ca 0. Dacă şirul curent este

mai mic decât şirul dat ca parametru, metoda scoate un rezultat mai mic ca 0.

Exemplu:

```
String s1="Abc";
String s2="Aa";
int rez=s1.compareTo(s2); // >0
```

5. Vectori de şiruri.

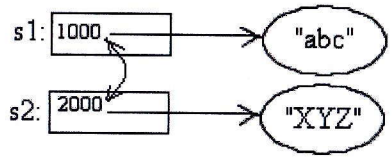
Vectorii se şiruri se declară la fel ca si vectorii de primitive.

Exemplu

Se citesc N nume de la tastatură într-un vector de stringuri. Să se sorteze vectorul de stringuri în ordine alfabetică.

```
class VectorS
{
 public static void main(String args[ ])
 {
  int N=Integer.parseInt(JOptionPane.showInputDialog("N="));
  String nume[ ]=new String[N];
  for (int i=0; i<N; i++)
   nume[i]= JOptionPane.showInputDialog("nume=");
  sortare(nume);
  for (int i=0; i<N; i++)
   System.out.println(nume[i]);
 }
 private static void sortare(String nume[ ])
 {
  for (int i=0; i<nume.length( )-1; i++)
   for (int j=i+1; j<nume.length( ); j++)
    //compar pe nume[i] cu nume[j]:
    if (nume[i].compareTo(nume[j])>0){
      String aux=nume[i];
      nume[i]=nume[j];
      nume[j]=aux; }
    }
 }
```

OBS: Când interschimbăm două şiruri, se interschimbă doar referinţele lor.



6. Clasa StringTokenizer

Pentru a extrage atomii lexicali dintr-un string, se foloseşte clasa **StringTokenizer,** definită în pachetul **java.util.**

Constructori:

public StringTokenizer(String s)

Este folosit atunci când separatorul implicit dintre atomi este caracterul spaţiu.

public StringTokenizer(String s, String separator)

Acest constructor se foloseste cand folosim alt caracter ca separator (diferit de spatiu), caracter ce va fi dat ca parametru.

Exemplu:In stringul "23:20:15", caracterul de separaţie este :

Metode:

a) **public int countTokens( )**

Returnează numărul de atomi din string.

b) **public String nextToken( )**

Returnează atomul curent din string.

c) **public boolean hasMoreTokens( )**

Returnează *true* dacă mai sunt atomi neextraşi din şir.

Exemplu

1. Se citesc de la tastatură sub formă de şiruri de caractere doi timpi, în formatul hh:mm:ss (ore: minute: secunde). Să se afişeze care timp este mai mare.

Exemplu:

T1= 5:35:42

T2= 5:18:50

Se va afişa: T2 > T1

```
import javax.swing.*;
import java.util.*;
class ComparaTimpi
{
 public static void main(String args[])
 {
  String timp1=JOptionPane.showInputDialog
```

```
                ("timp1 (hh:mm:ss) = ");
String timp2=JOptionPane.showInputDialog
                ("timp2 (hh:mm:ss) = ");
//Extragem din fiecare timp, orele, minutele si secundele.
// Folosim clasa StringTokenizer pentru a extrage acesti atomi.
StringTokenizer tk=new StringTokenizer(timp1,":");
int ore1=Integer.parseInt(tk.nextToken());
int min1=Integer.parseInt(tk.nextToken());
int sec1=Integer.parseInt(tk.nextToken());
//Calculam primul timp, in secunde:
int T1=3600*ore1+60*min1+sec1;
//Similar, pentru timp2 :
tk=new StringTokenizer(timp2,":");
int ore2=Integer.parseInt(tk.nextToken());
int min2=Integer.parseInt(tk.nextToken());
int sec2=Integer.parseInt(tk.nextToken());
int T2=3600*ore2+60*min2+sec2;
if(T1>T2)System.out.println("timp1 > timp2");
else if(T1==T2)System.out.println("timp1 = timp2");
else System.out.println("timp1 < timp2");
}
}
```


Capitolul 6
Moştenirea

1. **Generalităţi.**
2. **Exemple.**

1. Generalităţi.

Moştenirea este capacitatea prin care dezvoltăm o nouă clasă plecând de la o clasă existentă. Noua clasă obţinută prin moştenire de la clasa existentă, se cheamă **clasă derivată** (sau **clasă fiu** sau **subclasă**). Clasa existentă din care prin moştenire derivăm subclasa se cheamă **clasă de bază** (sau **clasă părinte** sau **supraclasă**).

Moştenirea este o trăsătură fundamentală în POO. Ea permite să dezvoltăm mai rapid noi clase pe baza claselor existente. Astfel, în loc să proiectăm de la zero o nouă clasă, căutăm între clasele existente o clasă asemănătoare şi noua clasă o derivăm din aceasta. Acest stil de programare se cheamă **programare prin diferenţe** (în noua clasă programam doar diferenţele faţă de clasa existentă).

Subclasarea unor clase existente se face pentru a nu "reinventa" cod deja creat.

În biblioteca grafică în pachetul **javax.swing** există **clasa JFrame,** ce modelează o fereastră grafica (cu bara de titlu, cu cele trei butoane tipice si cu comportament de fereastră). Dacă vrem să facem o aplicaţie în care într-o fereastră avem mai multe componente grafice, atunci clasa necesară în această aplicaţie va subclasa (va moşteni) clasa JFrame.

Pentru a implementa moştenirea se foloseşte cuvântul cheie **extends**.

Exemplu

class FereastraTestGrilă extends JFrame

Legat de moştenire există un specificator de acces ce se aplică membrilor unei clase (variabile de instanţă, constructori, metode) şi anume specificatorul de acces **protected**.

Membrii unei superclase declaraţi **protected** pot fi accesaţi doar de subclasele ei. Aceste subclase pot face parte din alte package-uri de clase decât package-ul clasei de bază.

Clasele ce fac parte din acelaşi package ca şi superclasa, chiar dacă nu o subclasează (nu o moştenesc) au acces şi la membrii protected.

În mod evident, la membrii **private** nu au acces nici subclasele.

În Java, spre deosebire de limbajul C++, nu există **moştenire multiplă**, adică în Java o subclasă nu poate avea decât un singur părinte. În Java, o clasă extinde o singură clasă (nu moşteneşte decât de la o singură clasă), dar poate să implementeze mai multe **interfeţe**.

Subclasa moşteneşte de la superclasă membrii **protected** sau **public** ai superclasei.

În subclasă pot fi adăugate noi variabile de instanţă(care nu există în superclasă) şi noi metode sau subclasa poate să redefinească anumite metode moştenite de la superclasă.

Constructorii nu sunt moşteniţi de la superclasă la subclasă.

Din subclasă se poate apela constructorul clasei de bază folosind cuvântul cheie **super(...)**, cu parametrii necesari.

În cazul în care se apelează cu **super(...)** constructorul clasei de bază, această instrucţiune trebuie să fie prima instrucţiune din metoda respectivă (ce conţine apelul cu **super**).

Strămoşul tuturor claselor din Java este **clasa Object**.

Chiar dacă o clasă nu extinde în mod explicit o altă clasă (nu foloseşte cuvântul cheie *extends*), ea moşteneşte în mod implicit clasa Object.

În clasa Object sunt definite metode ce sunt moştenite de toate clasele din Java. Iata cateva din acestea:

- **metoda equals( )**

public boolean equals(Object o)

Se foloseşte pentru a compara două obiecte din punct de vedere al egalităţii referinţelor.

Multe clase Java, redefinesc metoda equals() moştenită de la clasa Object, pentru a compara două obiecte din punct de vedere al conţinutului. Astfel, în clasa String, metoda equals() este redefinită. Ea returnează *true*, dacă cele două şiruri comparate au acelaşi conţinut.

Exemplu:

```
String s1="abc";
String s2=new String(s1);
if(s1.equals(s2))System.out.println("s1 si s2 au acelasi continut.");
else System.out.println("s1 si s2 nu au acelasi continut.");
//Se va afisa:
// s1 si s2 au acelasi continut.
```


- **metoda toString( )**

public string toString( )

Se foloseşte pentru a da o reprezentare sub formă de String unui obiect.

2. Exemple.

Exemplul 1

Să se construiască clasa ContBancar, folosită pentru a modela un cont bancar, ce are ca variabilă de instanţă privată, variabila *suma*, (suma de bani din cont). Ca metode:

- constructorul;
- adauga(), ce are ca parametru un număr real x, valoarea ce se adaugă în cont;
- extrage(), ce are ca parametru un număr real x, valoarea ce se extrage din cont, şi care scoate ca rezultat *true*, dacă se poate face extragerea (suma >= x), şi *false* în caz contrar;
- getSuma(), ce returnează valoarea variabilei private suma;
- afisare(), ce afişează valoarea sumei de bani din cont.

Pe baza clasei ContBancar se va dezvolta prin derivare (moştenire) clasa ContBancarExtins, în care se va adăuga o nouă variabilă de instanţă: rata dobânzii anuale şi o nouă metodă: adaugaDobandaLunara(), ce adaugă în cont dobânda calculată după trecerea unei luni. În clasa ContBancarExtins se va redefini şi metoda afisare(), astfel încât să se afişeze şi rata dobânzii. De asemenea, în această nouă clasă se va defini constructorul, prin care se iniţializează suma de bani din cont şi rata dobânzii.

Să se scrie şi o clasă de test pentru clasa ContBancarExtins.

```
class ContBancar
{
 private double suma;
 public ContBancar(double S)
 {
        suma=S;
 }
 public void adauga(double S)
 {
        suma=suma+S;
 }
 public boolean extrage(double S)
```

```
{
        if(S>suma)return false;
        suma=suma-S;
        return true;
}
public double getSuma()
{
        return suma;
}
public void afisare()
{
        System.out.println("suma="+suma);
}
}

class ContBancarExtins extends ContBancar
{
private double rd;//rata dobanzii anuale
public ContBancarExtins(double S,double rata)
{
        this.S=S;
        super(S);
        rd=rata;
}
public void adaugaDobandaLunara()
{
        double S=this.getSuma();
        double dobanda=S*rd/12;
        this.adauga(dobanda);
}
public void afisare()
{
        System.out.println("suma="+this.getSuma());
        System.out.println("rata dobanzii="+rd);
}
}

class TestCont
{
public static void main(String args[])
```



```
{
        ContBancarExtins c=new ContBancarExtins(1000,0.12);
        c.adauga(1000);
        c.adaugaDobandaLunara();
        c.afisare();
}
}
```

Exemplul 2

Să se construiască clasa Cerc, ce are ca variabilă de instanţă privată, un număr întreg r, ce reprezintă raza unui cerc. În această clasă avem ca metode:

- constructorul, ce face iniţializarea razei;
- getRaza(), ce returnează raza;
- calculArie(),ce returnează aria cercului;
- suntEgale(), ce are ca parametru un Cerc c, şi care returnează *true* dacă cercul curent este egal cu cercul c (au aceeaşi rază).
- afisare(), ce afişează raza cercului.

Din clasa Cerc se va deriva clasa CercExtins, în care se vor adăuga ca variabile de instanţă x şi y: coordonatele centrului şi se vor redefini metodele suntEgale() (cercurile sunt egale când au aceeaşi rază şi aceleaşi coordonate ale centrului), şi afisare() (pe lângă rază, va afişa şi coordonatele centrului)

Scriem şi o clasă de test pentru clasa CercExtins.

```
class Cerc
{
 private int raza;
 public Cerc(int x)
 {
  raza=x;
 }
 public int getRaza()
 {
  return raza;
 }
 public double calculArie()
 {
  return Math.PI*raza*raza;
 }
```

```
public boolean suntEgale(Cerc c)
 {
  if(this.raza==c.raza)return true;
  else return false;
 }
public void afisare()
 {
  System.out.println("raza="+raza);
}

}
class CercExtins extends Cerc
{
 private int x;
 private int y;
 public CercExtins(int r,int x0, int y0 )
 {
  super(r);
  x=x0;
  y=y0;
 }
 public boolean suntEgale(CercExtins c)
{
 if((this.getRaza()==c.getRaza())&&(this.x==c.x)&&(this.y==c.y))
 return true;
 else return false;
}
 public void afisare()
{
 System.out.println("raza="+this.getRaza());
 System.out.println("x="+x);
 System.out.println("y="+y);
 }
}

class TestCercExtins
{
 public static void main (String args[])
 {
  CercExtins c=new CercExtins(3,0,1);
```



```
  System.out.println("Aria= "+c.calculArie());
  CercExtins c1=new CercExtins(3,0,10);
  System.out.println("Sunt egale= "+c.suntEgale(c1));
 }
}
```

Exemplul 3

Să se construiască clasa Punct3D, folosită pentru a modela un punct în spaţiu, ce are ca variabile de instanţă x, y, z, coordonatele unui punct în spaţiu. Ca metode:

- constructorul;
- muta(), ce are trei parametrii dx, dy şi dz, pe baza cărora noile coordonate ale punctului devin: x+dx, y+dy, z+dz;
- compara(), ce are ca parametru un punct p, şi care returnează *true*, dacă punctul curent (cel referit prin *this*) este egal cu punctul p, şi *false* în caz contrar;
- distanta(), ce are ca parametru un punct p, şi care returnează distanţa între punctul curent şi punctul p;
- getX() ce returnează valoarea coordonatei x;
- getY() ce returnează valoarea coordonatei y;
- getZ() ce returnează valoarea coordonatei z;
- afisare() ce afişează coordonatele punctului.

Pe baza clasei Punct3D, se va dezvolta clasa Punct3DColor, în care se va adăuga o nouă variabilă de instanţă de tipul String: culoarea punctului şi o nouă metodă getCuloare() ce returnează culoarea punctului. Se vor redefini metodele compara() şi afişare() şi noul constructor.

Să se scrie şi o clasă de test pentru clasa Punct3DColor.

```
class Punct3D
{
 private int x;//coordonata x a punctului
 private int y;
 private int z;
 public Punct3D(int x,int y, int z )
 {
   this.x=x;
   this.y=y;
   this.z=z;
 }
 public int getX()
```

```
{
  return x;
}
public int getY()
{
        return y;
}
public int getZ()
{
        return z;
}
public void afisare()
{
        System.out.println("x="+x);
        System.out.println("y="+y);
        System.out.println("z="+z);
}
public void muta(int dx, int dy, int dz)
{
        x=x+dx;
        y=y+dy;
        z=z+dz;
}
public boolean compara(Punct3D p)
{
        if((x==p.x)&&(y==p.y)&&(z==p.z))
          return true;
        else return false;
}
public double distanta(Punct3D p)
{
        double dx=this.x-p.x;
        double dy=this.y-p.y;
        double dz=this.z-p.z;
        double dist=Math.sqrt(dx*dx+dy*dy+dz*dz);
        return dist;
}
}
```



```
class Punct3DColor extends Punct3D
{
 private String culoare;
 public Punct3DColor(int x, int y, int z, String culoare)
 {
  super(x,y,z);
  this.culoare=culoare;
 }
 public String getCuloare()
 {
        return culoare;
 }

 public void afisare()
 {
        System.out.println("x="+getX());
        System.out.println("y="+getY());
        System.out.println("z="+getZ());
        System.out.println("culoare="+culoare);
 }
 public boolean compara(Punct3DColor p)
 {
        if((this.getX()==p.getX())&&
          (this.getY()==p.getY())&&
          (this.getZ()==p.getZ())&&
          (this.culoare==p.culoare))
         return true;
        else return false;
 }
}

class TestPuncte
{
 public static void main (String args[])
 {
        Punct3DColor p=new Punct3DColor(0,1,2,"negru");
        p.muta(1,1,1);
        p.afisare();
 }
}
```

Exemplul 4

Să se dezvolte clasa Persoana ce are ca variabile de instanţă numele şi prenumele unei persoane şi vârsta ei, şi ca metode:

- constructorul ce face iniţializările;
- getNume(), ce returnează numele;
- getPrenume(), ce returnează prenumele;
- afisare(), ce afişează informaţiile despre persoană.

Din clasa Persoană se va deriva clasa Student, ce are în plus ca variabile de instanţă, numele facultaţii pe care o urmează şi numărul matricol. În clasa Student se va dezvolta un nou constructor şi se va redefini metoda afisare(). Se vor adăuga în plus metodele:

- getFacultate();
- getNumărMatricol().

Se va dezvolta o aplicaţie în care se vor citi de la tastatură N= 10 studenţi, ce se vor memora într-un vector. Se vor afişa câţi studenţi au prenumele "Ion".

```
import javax.swing.*;
class Persoana
{
 private String nume;
 private String prenume;
 private int varsta;
 public Persoana(String n, String p, int v)
 {
  nume=n;
  prenume=p;
  varsta=v;
 }
 public String getNume()
 {
 return nume;
 }
 public String getPrenume()
 {
 return prenume;
 }
 public void afisare()
 {
 System.out.println(nume+" "+prenume+" : "+varsta);
```



```
 }
}
class Student extends Persoana
{
 private String numeFacultate;
 private int nrMatricol;
 public Student(String n, String p, int v, String facult, int nrMatr)
 {
  super(n,p,v);
  numeFacultate=facult;
  nrMatricol=nrMatr;
 }
 public String getFacultate()
 {
 return numeFacultate;
 }
 public int getNumarMatricol()
 {
 return nrMatricol;
 }
}
class TestStudenti
{
 public static void main(String args[])
 {
  final int N=2;
  int i;
  Student s[]=new Student[N];
  for(i=0;i<N;i++){
   String nume=JOptionPane.showInputDialog("nume=");
   String prenume=JOptionPane.showInputDialog("prenume=");
   int varsta=
  Integer.parseInt(JOptionPane.showInputDialog("varsta="));
  String facultate=JOptionPane.showInputDialog("facultate=");
  int nrMatr=Integer.parseInt
  (JOptionPane.showInputDialog("nr. matricol="));
  s[i]=new Student(nume,prenume,varsta,facultate,nrMatr);
 }
 int contor_ion=0;
for(i=0;i<N;i++){
```

```
 String prenumeCrt=s[i].getPrenume();
 if(prenumeCrt.compareTo("Ion")==0)contor_ion++;
 }
System.out.println(contor_ion);
}
}
```


Capitolul 7
Clase abstracte. Interfeţe

1. Clase abstracte.
2. Interfeţe

1. Clase abstracte.

O clasă abstractă este o clasă din care nu se pot instanţia obiecte. Din ea se pot însă deriva noi clase (poate fi subclasată). Ea este folositoare ca şi **şablon** moştenit de alte clase.

Clasa abstractă poate să conţină două feluri de metode: metode care au definiţia completă (metode care au şi implementare) şi metode care nu sunt implementate(nu au corp, au doar antet). Metodele care nu sunt implementate se definesc cu ajutorul cuvântului cheie **abstract**. În general, aceste metode abstracte sunt implementate de către clasele derivate din clasa abstractă.

O clasă abstractă se defineşte cu ajutorul cuvântului cheie **abstract**.

Exemplu

```
abstract class A
{
   ..............
}
A obj=new A(); //eroare de sintaxă!!!
```

Exemplu

Definim clasa abstractă A ce are două metode: una implementată şi alta care nu este implementată.

- metoda abstractă calcul();
- metoda durataCalcul() ce returnează durata exprimată în milisecunde, a execuţiei metodei calcul();

Din clasa abstractă A, se va deriva clasa B ce conţine implementarea metodei calcul(). Se va dezvolta şi o clasă de test, pentru clasa derivată B.

```
import javax.swing.*;
import java.util.*;
abstract class A
{
```

```
  abstract public void calcul(int N);
  public long durataCalcul(int N){
  long t1=System.currentTimeMillis();
  calcul(N);
  long t2=System.currentTimeMillis();
  return (t2-t1);
 }
}
class B extends A
{
public void calcul(int N)
{
 //Calculeaza N*N*N produse
 int i,j,k;
 long rezultat;
 for(i=1;i<=N;i++)
  for(j=1;j<=N;j++)
    for(k=1;k<=N;k++)
       rezultat=i*j*k;
 }
}
class Test
{
 public static void main(String args[])
 {
  final int N=1000;
  B b=new B();
  System.out.println("durata calcul = "+b.durataCalcul(N)+" ms.");
 }
}
```

2. Interfețe.

O interfață conține o listă de semnături de metode (metode fără implementare). O interfață poate să definească, de asemenea, constante. O clasă ce implementează o anumită interfață se obligă să dea implementarea pentru **toate** metodele declarate în interfață.

O clasă ce implementează o anumită interfață folosește cuvântul cheie **implements**.

Noțiunea de interfață poate fi văzută ca o generalizare a noțiunii de clasă abstractă, în sensul că toate metodele din interfață sunt abstracte(fără implementare).

O interfață se defineşte cu ajutorul cuvântului cheie **interface**.

O clasă nu poate să moştenească decât o singură clasă, dar poate implementa mai multe interfețe.

De ce s-a inventat şi noțiunea de interfață? S-a constatat că în timp, semnăturile metodelor sunt mai viabile, în sensul că "au o viață mai lungă" decât implementările.

Exemplu

Definim interfața Forma în care avem două antete de metode: arieTotală( ) şi volum( ).

```
interface Forma
{
 public double arieTotală( );
 public double volum( );
}
```

Definim două clase ce implementează această interfață:

- clasa Cub
- clasa Paralelipiped

```
class Cub implements Forma
{
 private double a;
 public Cub(int a)
 {
   this.a=a;
 }
 public double getA( )
 {
   return a;
 }
 public double arieTotală( )
 {
   return 6*a*a;
 }
 public double volum( )
 {
   return a*a*a;
```

```
  }
}

class Paralelipiped implements Forma
{
 private double x;
 private double y;
 private double z;

 public Paralelipiped(int a, int b, int c)
 {
  x=a;y=b;z=c;
 }
 public double arieTotală( )
 {
   return 2*(x*y+x*z+y*z);
 }
 public double volum( )
 {
   return x*y*z;
  }
}
```


Capitolul 8
Polimorfism

1. **Generalități.**
2. **Exemple de polimorfism la execuție.**

1. Generalități.

Polimorfismul este de două tipuri:

- polimorfism la compilare (polimorfism parametric)
- polimorfism la execuție.

În cazul polimorfismului parametric (în aceeaşi clasă mai multe metode cu acelaşi nume, dar cu număr diferit de parametrii sau cu tipuri diferite de parametrii) legarea codului metodei adecvate se face în faza de compilare **(early-binding)**.

Exemplu

```
class ExempluPolimorfismParametric
{
 public static void main(String args[ ])
 {
  int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
  int b=Integer.parseInt(JOptionPane.showInputDialog("b="));
  int c=Integer.parseInt(JOptionPane.showInputDialog("c="));
  int max1=maxim(a,b);
  System.out.println("max1="+max1);
  int max2=maxim(a,b,c);
  System.out.println("max2="+max2);
 }
 private static int maxim(int a, int b)
 {
  if (a>b) return a;
  else return b;
  }
  private static int maxim(int a, int b, int c)
 {
  int rez=a;
  if (b>rez) rez=b;
  if (c>rez) rez=c;
  return rez;
  }
```

}

Noțiunea de **polimorfism în faza de execuție** este legată in primul rand de noțiunea de moştenire.

În cazul unei metode prezentă în clasa de bază şi redefinită în clasa derivată (acelaşi nume, două forme: o formă în clasa de bază, o formă în clasa derivată) în anumite situații, numai în faza de execuție se poate şti care formă de metodă se va apela. Altfel spus, numai la execuție se poate şti care cod va fi folosit(va fi "legat") de aplicație.

Codul metodei adecvate va fi "legat" la execuție şi aceasta se cheamă **late-binding** (legare târzie), spre deosebire de legarea codului din faza de compilare **(early-binding)**.

2. Exemple de polimorfism la executie.

Exemplul 1

Să se construiască clasa Punct ce are ca variabile de instanță două numere întregi x şi y – coordonatele unui punct în plan, şi ca metode:

- Constructorul ce face inițializările;
- getX() ce returnează valoarea coordonatei x
- getY() ce returnează valoarea coordonatei y
- afisare() în care se afişează coordonatele punctului din clasa Punct

Din clasa Punct se derivează două clase: PunctColor şi Punct3D.

Clasa PunctColor față de clasa Punct are în plus o variabilă de instanță în care este memorată culoarea punctului, un nou constructor în care este inițializată şi culoarea, metoda getCuloare() ce returnează culoarea, şi redefineşte metoda clasei de bază, afişare(), afişând pe lângă coordonatele x şi y şi culoarea.

Clasa Punct3D, ce reprezintă un punct în spațiu, față de clasa Punct are în plus o variabilă de instanță z, un nou constructor în care sunt inițializate toate cele trei coordonate, metoda getZ() ce returnează valoarea coordonatei z, şi redefineşte metoda clasei de bază, afişare(), afişând pe lângă coordonatele x şi y şi coordonata z.

Folosind aceste trei clase se va dezvolta o aplicație în care se vor citi de la tastatură N puncte (N- citit anterior), de tipul PunctColor sau Punct3D. Pentru fiecare punct, în momentul citirii utilizatorul aplicației va specifica dacă va introduce un PunctColor sau un Punct3D. Cele N puncte se vor memora într-un vector de obiecte de

tipul Punct (clasa de bază). În final se vor afişa pentru fiecare punct din cele N informațiile memorate (pentru fiecare punct se va apela metoda afişare()).

Această aplicație ilustrează conceptul de polimorfism. Compilatorul ştie la rulare ce versiune de metodă afişare() să apeleze.

```
import javax.swing.*;
class Punct
{
 private int x;//coordonata x a punctului
 private int y;
 public Punct(int x0, int y0)
 {
  x=x0;
  y=y0;
 }
 public int getX()
 {
  return x;
 }
 public int getY()
 {
  return y;
 }
 public void afisare()
 {
  System.out.println("x="+x);
  System.out.println("y="+y);
 }
}

class PunctColor extends Punct
{
 private String culoare;
 public PunctColor(int x, int y, String culoare)
 {
  super(x,y);
  this.culoare=culoare;
 }
 public String getCuloare()
 {
```

```
 return culoare;
}

public void afisare()
{
 //System.out.println("x="+x);GRESIT! x - este var. privata !
 System.out.println("x="+getX());
 System.out.println("y="+getY());;
 System.out.println("culoare="+culoare);
}
}

class Punct3D extends Punct
{
 private int z;
 public Punct3D(int x,int y, int z )
 {
  super(x,y);
  this.z=z;
 }
 public int getZ()
 {
  return z;
 }
 public void afisare()
 {
  System.out.println("x="+getX());
  System.out.println("y="+getY());
  System.out.println("z="+z);
 }
}

class AfisarePuncte
{
  public static void main (String args[])
  {
   int N;//numarul de puncte
   N=Integer.parseInt(JOptionPane.showInputDialog("N="));
   Punct p[]=new Punct[N];//vectorul de obiecte Punct (clasa de
   //baza)
```



```
   int i;
  for(i=0;i<N;i++){
   String sRaspuns=JOptionPane.showInputDialog
                    ("Tip punct:  (0,1,2) : ");
   int raspuns=Integer.parseInt(sRaspuns);
   int x=Integer.parseInt(JOptionPane.showInputDialog("x="));
   int y=Integer.parseInt(JOptionPane.showInputDialog("y="));
    if(raspuns==0){
     //citeste un Punct:
     p[i]=new Punct(x,y);}
    else if(raspuns==1){
     //citeste un PunctColor:
     String culoare=JOptionPane.showInputDialog("culoare=");
     p[i]=new PunctColor(x,y,culoare);}
    else if(raspuns==2){
     //citeste un Punct3D:
     int z=Integer.parseInt(JOptionPane.showInputDialog("z="));
     p[i]=new Punct3D(x,y,z);}
   }//for
   //Afisare vector:
   for(i=0;i<N;i++)
     p[i].afisare();
}
}
```

Exemplul 2

1. Să construiască clasa Număr ce are ca variabilă de instanță un număr întreg a, şi ca metode:
   - constructorul;
   - afisare().

Din clasa Numar, se va deriva clasa DouaNumere, în care se va adăuga variabila de instanță b (ce reprezintă al doilea număr), şi se va modifica constructorul şi metoda afisare().

Folosind cele două clase, se va dezvolta o aplicație în care se generează un număr aleator 0 sau 1. Dacă este 0, se va instanția un obiect din clasa Numar (prin citire de la tastatură), dacă este 1, se va instanția un obiect din clasa DouăNumere (tot prin citire de la tastatură). Pentru obiectul instanțiat se va apela metoda afisare().

```
import java.util.*;
import javax.swing.*;
class Numar
{
 private int a;
 public Numar(int x)
 {
  a=x;
 }
 public void afisare()
 {
  System.out.println("a="+a);
 }
}

class DouaNumere extends Numar
{
 private int b;
 public DouaNumere(int a,int b )
 {
  super(a);
  this.b=b;
 }
 public void afisare()
 {
  //afisare a:
  super.afisare();//Cu cuvantul cheie super se apeleaza metoda
                  //clasei de baza
  //afisare b:
  System.out.println("b="+b);
 }
}

class AfisareNumere
{
  public static void main (String args[])
  {
   Numar n;
   Random r=new Random();
   int caz=r.nextInt(2);
```



```
  if(caz==0){
  //citeste un numar:
    int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
    n=new Numar(a);
  }else
  {
    //citeste doua numere:
    int a=Integer.parseInt(JOptionPane.showInputDialog("a="));
    int b=Integer.parseInt(JOptionPane.showInputDialog("b="));
    n=new DouaNumere(a,b);
  }
   //Afisare:
   n. afisare();//Numai la executie se stie care versiune de metoda
                // afisare() se va apela
 }
}
```

Exemplul 3

Folosind cele două clase anterioare, Cerc şi CercExtins create în capitolul 6, să se dezvolte o aplicație în care se vor citi N cercuri (de tipul Cerc sau CercExtins), ce se memorează într-un vector. Citirea unui obiect de tip Cerc sau CercExtins este dată de valoarea 0 sau 1 a unui număr aleator generat.

```
import java.util.*;
import javax.swing.*;
class Cerc
{
 private int raza;
 public Cerc(int x)
 {
  raza=x;
 }
public void afisare()
 {
  System.out.println("raza="+raza);
 }

}
class CercExtins extends Cerc
{
```

```
 private int x;
 private int y;
 public CercExtins(int r,int x0, int y0 )
 {
  super(r);
  x=x0;
  y=y0;
}

public void afisare()
{
 System.out.println("raza="+this.getRaza());
 System.out.println("x="+x);
 System.out.println("y="+y);
 }
}

class AfisareCercuri
{
  public static void main (String args[])
  {
    final int N=3;//numarul de cercuri
    Cerc c[]=new Cerc[N];//vectorul de obiecte Cerc (clasa de
                         //baza)
    Random r=new Random();
    int i;
    for(i=0;i<N;i++){
     int caz=r.nextInt(2);
     if(caz==0){
       //citeste un Cerc:
     String s_raza=JOptionPane.showInputDialog("raza=");
     c[i]=new Cerc(Integer.parseInt(s_raza));
     }else{
           //citeste un CercExins:
           String s_raza;
           s_raza=JOptionPane.showInputDialog("raza=");
           String s_x=JOptionPane.showInputDialog("x=");
           String s_y=JOptionPane.showInputDialog("y=");
           c[i]=new CercExtins(Integer.parseInt(s_raza),
                               Integer.parseInt(s_x),
```



```
                               Integer.parseInt(s_y));
      }
     }//for
    //Afisare vector:
    for(i=0;i<N;i++)
     c[i].afisare();
  }
  }
```

Exemplul 4

(Polimorfism la execuție, în cazul folosirii interfețelor)

```
interface Forma
{
  public double arieTotală( );
  public double volum( );
}

class Cub implements Forma
{
 private double a;
 public double arieTotala()
 {
  return 6*a*a;
 }
 public double volum( )
 {
  return a*a*a;
 }
}

class Paralelipiped implements Forma
{
 private double l, w, h;
 public double arieTotală( )
 {
  return 2*l*w+2*l*h+2*w*h;
 }
 public double volum( )
 {
```

```
 return l*w*h;
}
}

class ExempluPolimorfismCuInterfețe
{
 public static void main(String args[ ])
 {
  Forma f[ ]={new Cub(10), new Paralelipiped(1,2,7)};
  for(int i=0;i<f.length;i++)
    System.out.println("Obiectul din clasa"+f[i].getClass( )+
                       "are volumul"+f[i].volum( ));
}
}
```


Capitolul 9
Excepții

1. Generalitati.
2. Blocuri try-catch.

1. Generalitati.

La executarea anumitor metode pot apărea erori, denumite în Java **excepții**. Mecanismul de tratare a acestor erori în Java, este diferit de mecanismul de tratare a erorilor în programarea clasică.

În programarea clasică erorile erau semnalate prin intermediul unor argumente ale funcției sau printr-un rezultat scos de instructiunea return.

Exemplu

In limbajul C:

Scriem o funcție ce returneaza rezultatul împărțirii a două numere întregi a şi b.

```
void  împărțire(int a, int b, double& rezultat, int& esteEroare)
{
  if (b= =0){
    esteEroare=1;
    return; }
  esteEroare=0;
  rezultat=a/(double)b;
}
```

Ca dezavantaj principal al mecanismului de tratare a erorilor în programarea clasică:

- se poate uita de tratarea erorilor
- dacă nu se uită, totuşi se încarcă antetul funcției cu parametrii suplimentari.

Consecința cea mai gravă o are însă uitarea tratării erorilor (compilatorul, în limbajele clasice nu ne forțează să tratăm erorile).

Exemplu:

```
double împărțire(int a, int b)
{
 return a/(double)b;
 }
//Exemplu de folosire:
 rez=împărțire(13,0); //compilatorul nu semnalează eroarea!
```

Astfel, in programarea clasică, la fişiere, compilatorul nu ne forţează să tratăm cazul în care nu există fişierul ce trebuie prelucrat.

În Java, erorile ce apar în execuţia unei metode sunt denumite **excepţii**. Excepţiile sunt obiecte derivate (care moştenesc) dintr-una din clasele de bibliotecă:

a) Exception

b) RunTimeException

Excepţiile din clasa **RunTimeException** nu sunt obligatoriu de tratat. Cele din clasa **Exception** sunt obligatoriu de tratat.

Tratarea excepţiilor se face în blocuri **try...catch**.

Atunci când în executarea metodei apare o situaţie anormală (eroarea), "se aruncă" un obiect excepţie, de tipul declarat în definirea metodei. O metodă în care se generează o excepţie şi care "aruncă" o excepţie, trebuie să aibă în antetul ei cuvântul cheie **"throws"** urmat de tipul excepţiei.

Exemplu

Metoda **parseInt( )** din **clasa Integer**, poate să genereze o excepţie de tipul **NumberFormatException** (subclasă a lui **RunTimeException**), excepţie care nu este obligatoriu de tratat.

```
String s="123";
int nr=Integer.parseInt(s);//Aici, nu este nici o exceptie.
int nr2=Integer.parseInt("12AB");//Aici, este exceptie.
```

În acest caz "se aruncă" o excepţie de tip **NumberFormatException**, neobligatoriu de tratat. Dacă ea nu a fost tratată de programator cu blocul **try...catch** se întrerupe anormal execuţia programului.

Avem următoarele categorii de excepţii:

- **excepţii aritmetice** (nu sunt obligatoriu de tratat)
  1. împărţirea cu 0
  2. radical dintr-un număr negativ
  3. depăşirea pentru un anumit tip de date

  Exemplu: byte x;
  x=1000; //eroare de depăşire
- **excepţii de adresare**

  Exemplu: apelarea unei metode folosind un obiect neinstanţiat, un obiect null.

  String s=null;
  .....
  int L=s.length( ); //eroare la execuţie
  Nu este o eroare obligatoriu de tratat.
- **excepţii în cazul vectorilor, indexare în afara graniţelor**

Nu este obligatoriu de tratat.

Se aruncă o excepţie de tip **ArrayIndexOutOfBoundsException**.

Exemplu:

```
int a[ ]=new int[10];
.....
a[20]=7; //nu este eroare la compilare. Eroarea apare la execuţie.
```

Se opreşte programul forţat în acest punct, aruncându-se o excepţie de tipul **ArrayIndexOutOfBoundsException**.

În general, nu se recomandă tratarea excepţiilor care nu sunt obligatoriu de tratat pentru a nu încărca suplimentar codul programului cu blocuri **try...catch**.

- **excepţii legate de operaţiile de intrare-ieşire** (lucrul cu fişiere, operaţiile care se fac în reţea, lucrul cu baze de date).

  Aceste excepţii sunt excepţii obligatoriu de tratat.

  Dacă nu le tratăm (în blocuri **try...catch**), programul nu se compilează.

  În cazul fişierelor, cea mai des întâlnită excepţie este excepţia de tipul **IOException**, excepţie obligatoriu de tratat.

Exemplu:

În cazul programării în **C**, pentru deschiderea unui fişier text, avem următoarele instrucţiuni:

```
FILE* fp=fopen("date.txt", "rt");
//tratare eventuală eroare:
if (fp= =NULL) {
    .....
    exit(1); }
```

OBS: Această tratare nu este obligatorie.

În **Java** secvenţa corespunzătoare este următoarea:

```
FileReader fr=null;
try{
        fr=new FileReader("date.txt");
        .....
}catch(IOException e) {
        System.out.println(e);
        System.exit(1);
}
```

În Java este obligatoriu blocul **try...catch**.

2. Blocuri try-catch.

Sintaxa tipică pentru blocurile **try...catch** este următoarea:

```
try{
    // instrucţiuni ce pot genera excepţii.
}catch(tipExcepţie numeDeVariabila) {
    // instrucţiuni pentru tratarea excepţiei.
}
```

Exemplu

```
FileReader fr=null;
try{
    fr=new FileReader("date.txt");
    .....
}catch(IOException e) {
    System.out.println(e);
    System.exit(1);
}
```

Dacă în blocul **try...catch** nu apare nici o excepţie, se execută tot blocul **try**, nu se execută nici o instrucţiune din blocul **catch** şi apoi se continuă cu execuţia normală a programului (după blocul **try...catch**).

Dacă apare o excepţie, se părăseşte execuţia blocului **try** în locul unde a apărut excepţia şi se execută blocul **catch** corespunzător.

Cazul general de bloc **try...catch** este următorul:

```
try{
    // instrucţiuni
}catch(tipExcepţie e1) { //tratare excepţie e1 }
.....
catch(tipExcepţie en) { //tratare excepţie en }
finally { //instrucţiuni }
```

Instrucţiunile din blocul **finally** se execută în mod obligatoriu la finalul blocului **try...catch**, indiferent dacă a fost sau nu a fost excepţie. Dacă nu apare nici o excepţie, se execută tot blocul **try** şi apoi instrucţiunile din blocul **finally**.

Dacă apare o excepţie, se execută blocul **try** doar până în locul unde apare excepţia, se execută blocul **catch** corespunzător excepţiei apărute şi apoi se execută blocul **finally**.

Exemplu

Metoda **parseInt( )** din clasa **Integer** generează o excepţie care nu este obligatoriu de tratat de tipul **NumberFormatException**, atunci când argumentul ei este un String ce nu poate fi convertit la un număr întreg.

```
int n1=Integer.parseInt("123"); //nu este exceptie!
int n2=Integer.parseInt("12A*");//aici este excepţie
```

Exceptia este de tipul **NumberFormatException**, excepţie care nu este obligatoriu de tratat.

Definitia metodei **parseInt( )** în clasa **Integer** este următorul:

```
public static int parseInt(String s) throws
                                    NumberFormatException
{
    //codul metodei:
     .....
    // în caz de imposibilitate de a converti pe s la întreg:
    throw new NumberFormatException( );
     .....
}
```

Desi nu este obligatoriu, se poate trata excepţia **NumberFormatException,** astfel:

```
String s=JOptionPane.showInputDialog("nr=");
try{
     int nr=Integer.parseInt(s);
     System.out.println(nr);
}catch(NumberFormatException e) {
      System.out.println(e);
      System.exit(1);
}
```

O altă posibilitate de tratare a acestei excepţii, fără a ieşi din program, ci în caz de excepţie vom repeta citirea de la tastatură:

```
for(;;) {
 String s=JOptionPane.showInputDialog("nr=");
 boolean stop=true;
 try{
      int nr=Integer.parseInt(s);
      System.out.println(nr);
      // afişare divizori nr:
      for(int i=1;i<=nr;i++)
        if(nr%i==0) System.out.println(i);
```

```
}catch(NumberFormatException e)
{
 stop=false;
}
if(stop= =true) break;
}// for;;
```


Capitolul 10
Fişiere

1. Generalităţi.
2. Clase pentru fişiere text.

1. Generalităţi.

Fişierele sunt de două tipuri:

- **fişiere text**
- **fişiere binare**

Diferenţa principală dintre ele este dată de modul de codificare a informaţiei.

Exemplu:

Fie un fisier text ce contine ca informatie valoarea numerica : 259

Informaţia din fişierul text este codificată binar pe baza codurilor Unicode ale celor 3 caractere: '2', '5' si '9'.
Codul caracterului '2' este $50_{10}=2^5+2^4+2^1$=0 0 1 1.0 0 1 0
Deci, in binar: 00110010 .
Asemanator sunt codate si celelalte doua caractere: '5' si '9'.

Fie un fişier binar de numere întregi, ce conţine aceeasi informaţie:
259

Informatia va fi reprezentată ca număr întreg pe 4 octeţi, corespunzător reprezentării în baza 2 a numărului intreg 259:
$259_{10}=2^8+2^1+2^0$
Deci in baza 2:
00000000 00000000 00000001 00000011

O altă diferenţă între fişierele text şi fişierele binare este aceea ca fişierele text au înregistrări (linii) de lungime variabilă iar fişierele binare au înregistrări de lungime fixă. Astfel, intr-un fişier binar de numere întregi, fiecare înregistrare ocupă 4 octeţi.

Putem totuşi interpreta un fişier text ca un fişier binar, ce are ca înregistrare de bază octetul.

În Java, clasele care prelucrează fişiere se găsesc în pachetul **java.io**.

În cazul tuturor prelucrărilor de fişiere, se pot genera excepţii care sunt obligatoriu de tratat. Cea mai des întâlnită excepţie face parte din clasa **IOException**, excepţie obligatoriu de tratat.

În Java, clasele ce prelucrează fişiere sunt specializate, atât pentru tipurile de fişiere (text sau binar), cât şi pentru operaţiile ce se fac asupra fişierelor.

2. Clase pentru fisiere text.

a) Clasa FileReader.

Este o clasă folosită pentru a citi un fişier text caracter cu caracter.

Constructorul:

public FileReader(String numeFisier)

Metode:

a) **public int read( )** – citeşte şi returnează codul caracterului de pe poziţia curentă.
Determină avansarea în mod automat pe următoarea poziţie a capului de citire.
În caz de sfârşit de fişier returnează **-1**.

b) **public void close( )** – este folosită pentru a închide fişierul.

Exemplu: Se citeşte de la tastatură numele unui fişier text. Să afişăm dacă este prezent caracterul **a** în fişier.

```
class F1
{
 public static void main(String args[ ])
 {
  String numeF=JOptionPane.showInputDialog("nume fişier=");
  FileReader fr=null;
  boolean este_a=false;
  try{
  fr=new FileReader(numeF);
  //citesc fişierul caracter cu caracter:
  for(;;) {
        int cod=fr.read( )
;       if (cod== 1) break;
        if ((char)cod=='a') { este_a=true;
                              break; }
        }
```



```
  fr.close( );
  }catch(IOException e){
  System.out.println(e);
  System.exit(1); }
  if (este_a== true) System.out.println("Este prezent a");
  else System.out.println("Nu este prezent a");
 }
}
```

b) Clasa FileWriter.

Constructorul

public FileWriter(String numeFisier)

Metode:

public void write(char ch)

Este folosită pentru a scrie în fişier, pe poziţia curentă, caracterul dat ca parametru.

Exemplu: Citim de la tastatură o linie de text. Să copiem caracterele din acest text în fişierul "linie.txt" câte un caracter pe fiecare linie a fişierului text.

Exemplu: măr ⇒ m
ă
r

```
class F2
{
 public static void main(String args[ ])
 {
  String text=JOptionPane.showInputDialog("text=");
  FileWriter fw=null;
  try{
        fw=new FileWriter("linie.txt");
        for(int i=0; i<text.length( ); i++) {
                fw.write(text.charAt(i));
                fw.write('\n'); }
        fw.close( );
  }catch(IOException e){
                System.out.println(e);
                System.exit(1); }
 }//main
}//class
```

c) Clasa BufferedReader.

Este folosită pentru a citi un fişier text linie cu linie.

Constructorul:

public BufferedReader(FileReader fr)

Metode:

public String readLine( )

Returnează sub formă de String, linia curentă citită din fişier. În caz de sfârşit de fişier, returnează **null**.

Exemplul 1

Să se calculeze şi afişeze care este cea mai lungă linie din fişierul text "scrisoare.txt".

Algoritmul:

Iniţializăm maximul(lungimea celei mai lungi linii din fişier) cu 0.
Citim pe rând fiecare linie şi comparăm lungimea ei cu maximul.
Dacă este mai mare, schimbăm maximul.

```
class F3
{
 public static void main(String args[ ])
 {
  FileReader fr=null;
  BufferedReader bfr=null;
  int lMax=0;
  String linieMax="";//stringul vid
  try{
  fr=new FileReader("scrisoare.txt");
  bfr=new BufferedReader(fr);
  for(;;) {
        String s=bfr.readLine( );
        if (s= = null) break;
        int l=s.length( );
        if (l>lMax) { lMax=l;
                      linieMax=s; }
        }//for;;
  bfr.close( );
  fr.close( );
  }catch(IOException e){
        System.out.println(e);
        System.exit(1); }
```



```
  System.out.println(linieMax);
 }//main
}//class
```

Exemplul 2

Să se afişeze care este cel mai lung cuvânt dintr-un fişier text dat. Se consideră că în fişier nu se află cuvinte despărţite în silabe, la cap de rând.

```
import java.io.*;
import java.util.*;
class CelMaiLungCuvant
{
 public static void main(String args[])
 {
  FileReader fr=null;
  BufferedReader bfr=null;
  //initializari:
  String cuvantMax="";
  int lungimeMax=0;//initializare pt. dimensiunea celui mai lung
                   //cuvant
  try{
   fr=new FileReader("referat.txt");
   bfr=new BufferedReader(fr);
   for(;;){
    String linie=bfr.readLine();
    if(linie==null)break;//s-a terminat fisierul
    //Extragem cuvintele din aceasta linie:
    StringTokenizer tk=new StringTokenizer(linie);
    int n=tk.countTokens();
    for(int i=0;i<n;i++){
     String cuvant=tk.nextToken();
     int lungime=cuvant.length();
     if(lungime>lungimeMax){
        lungimeMax=lungime;
        cuvantMax=cuvant;
     }
    }
   }
   bfr.close();
   fr.close();
  }catch(IOException e){
```

```
    System.out.println(e);
    System.exit(1);}
 System.out.println("Cel mai lung cuvant este: "+cuvantMax);
 }//main
}
```

Exemplul 3

Se citeşte numele unui fişier text ce conţine mai multe linii. Să se afişeze dacă toate liniile sunt diferite între ele sau nu.

```
import java.io.*;
import javax.swing.*;
class LiniiDiferite
{
 public static void main(String args[])
 {
  String numeF=JOptionPane.showInputDialog("nume fisier=");
  FileReader fr=null;
  BufferedReader bfr=null;
  //Vom copia toate liniile din fisier, intr-un vector de Stringuri:
  String s[]=new String[1000];//dimensiune acoperitoare
  //initializare pt. numarul de linii din fisier:
  int N=0;
  try{
    fr=new FileReader(numeF);
    bfr=new BufferedReader(fr);
    for(;;){
     String linie=bfr.readLine();
     if(linie==null)break;//s-a terminat fisierul
     //copiem linia in vectorul s[]:
     s[N]=linie;
     N++;
    }
   bfr.close();
  fr.close();
 }catch(IOException e){
    System.out.println(e);
    System.exit(1);}
 //Verificam daca vectorul s[] are toate elementele diferite:
 for(int i=0;i<N-1;i++)
   for(int j=i+1;j<N;j++)
```



```
     if(s[i].compareTo(s[j])==0){
      System.out.println("Nu are toate liniile diferite !");
      return;}
  System.out.println("Are toate liniile diferite !");
  }//main
}
```

d) Clasa BufferedWriter.

Se foloseşte pentru a scrie un fişier text, linie cu linie .

Constructorul:

public BufferedWriter(FileWriter fw)

Metode:

a) public write(String s, int indexStart, int L)

Scrie in fisier, pe pozitia curenta, portiunea din sirul s (dat ca parametru), de lungime L caractere, incepand cu pozitia indexStart din sir.

b) public void newLine( )

Scrie un ENTER in fisier (trece pe linia următoare).

Exemplul 1

Se citesc N linii de la tastatura (N - cunoscut). Sa se copieze aceste linii intr-un fisier text.

```
import javax.swing.*;
import java.io.*;
class FisierScriePropozitii
{
 public static void main(String args[])
{
 FileWriter fw=null;
 BufferedWriter bfw=null;
 int N=Integer.parseInt(
 JOptionPane.showInputDialog("N="));
 try{
  fw=new FileWriter("propozitii.txt") ;
  bfw=new BufferedWriter(fw);
  for(int i=0;i<N;i++){
    String s=JOptionPane.showInputDialog("linie=");
    bfw.write(s,0,s.length());//index start si lungimea ce se scrie
    bfw.newLine();//scrie ENTER
```

```
  }
 bfw.close();
 fw.close();
}catch(IOException e){
                System.out.println("Eroare scriere fisier");
                System.exit(1);
}
}//main
}
```

Exemplul 2

Se da un fisier text de cuvinte (fiecare cuvant se afla pe o linie). Sa se construiasca un alt fisier text, ce contine cuvintele din primul fisier, sortate in ordine alfabetica.

```
import java.io.*;
class ScrieFisierSortat
{
 public static void main(String[] args)
 {
  FileReader fr=null;
  BufferedReader bfr=null;
  //Copiem toate cuvintele (liniile) intr-un vector de Stringuri:
  String st[]=new String[100];//dimensiune acoperitoare
  int nL=0;//numarul de linii din fisier
  try
  {
   fr=new FileReader("cuvinte.txt");
   bfr=new BufferedReader(fr);
   for (;;)
   {
    String s=bfr.readLine();
    if (s==null)break;
    else st[nL]=s;
    nL++;
   }
   bfr.close();
   fr.close();
  } catch(IOException e){
       System.out.println(e);
       System.exit(1);}
```



```
  //sortare bubblesort pentru vectorul st[]:
  for (;;)
   {
   boolean gata=true;
   for (int i=0;i<nL-1;i++)
    {
     if (st[i].compareTo(st[i+1])>0)
      {
       String aux=st[i];
       st[i]=st[i+1];
       st[i+1]=aux;
       gata=false;
      }
    }
    if (gata)break;
  }
  //copiere vector st[] in fisier:
  FileWriter fw=null;
  BufferedWriter bfw=null;
  try
   {
   fw=new FileWriter("cuvinteSortate.txt");
   bfw=new BufferedWriter(fw);
   for(int i=0;i<nL;i++){
   bfw.write(st[i],0,st[i].length());//index start si lungimea ce se scrie
   bfw.newLine();// scrie ENTER in fisier
   }
   bfw.close();
   fw.close();
  }catch(IOException e){
        System.out.println(e);
        System.exit(1);}
 }
}
```

Exemplul 3

Să se creeze prin program, un fişier text ce are următorul conţinut:

0 0 ... 0
1 1 ... 1

.....
9 9 ... 9
În fiecare linie din fişier sunt câte 10 numere separate prin spaţii.

```
import java.io.*;
class ScrieFisier
{
 public static void main(String[] args)
 {
  final int N=10; //numarul de linii din fisier
  FileWriter fw=null;
  BufferedWriter bfw=null;
  try
  {
   fw=new FileWriter("numere.txt");
   bfw=new BufferedWriter(fw);
   for(int i=0;i<N;i++){
    //Construim linia curenta:
    String s="";
     for(int j=0;j<N;j++)
       s=s+i+" ";
     bfw.write(s,0,s.length());
     bfw.newLine();// scrie ENTER in fisier
    }
    bfw.close();
    fw.close();
   }catch(IOException e){
         System.out.println(e);
         System.exit(1);}
 }
}
```


Capitolul 11
Grafica înJava

1. Generalităţi.
2. Exemple.

În realizarea aplicaţiilor grafice sunt implicate, în principal, pachetele de clase: **java.awt** si **javax.swing**.

În general, o aplicaţie grafică constă din definirea unei ferestre, fereastră în care programatorul aranjează componentele grafice.

Componentele grafice pot avea o reprezentare vizibilă sau pot să nu aibă o reprezentare vizibilă.

Exemple de componente grafice cu reprezentare vizibilă:

a. Butoane: fac parte din clasa **JButton**
b. Etichete: din clasa **JLabel**
c. Casete de validare : din clasa **JCheckBox**
d. Butoane radio: **c**lasa **JRadioButton**
e. Meniuri: clasele **JMenuBar**, **JMenuItem**.
f. Câmpuri de editare: clasa **JTextField**
g. Arie de text : clasa **JTextArea**

Fiecare clasă are mai multe metode.

Dintre componentele care nu au o reprezentare vizibilă ca exemplu tipic avem componentele de tip **container**. Astfel, clasa **JPanel** este un container intermediar, fără o reprezentare vizibilă. Rolul lui este de a aduna în acelaşi grup mai multe componente grafice, care pot fi manevrate ca un ansamblu..

Programarea grafică este o programare orientată pe evenimente. La realizarea aplicaţiei grafice cooperează programatorul cu mediul de execuţie Java.

Programatorul este cel care defineşte pentru aplicaţie care sunt sursele generatoare de evenimente si evenimentele de interes. Astfel, o sursă de eveniment poate să fie un buton, componentă de tipul **JButton**, şi evenimentul este declanşat atunci când utilizatorul face click cu mouse-ul pe acel buton.

O componentă grafică este desemnată de programator ca o sursă de evenimente, prin ataşarea de acea componentă a unui obiect de ascultare a evenimentului.

Ataşarea obiectului de ascultare a evenimentului se face printr-o metodă specifică. In cazul unui buton de tip **JButton**, ataşarea unui obiect de ascultare se face prin metoda **addActionListener( ).** Butonul respectiv devine astfel sursă de eveniment.

Nu orice componentă grafică poate fi sursă de evenimente. Un obiect de tipul **JLabel** nu poate să fie sursă de eveniment. De asemenea, nu toate componentele grafice care au potenţialul de a fi surse de evenimente, sunt desemnate de programator ca fiind surse de evenimente.

Programatorul trebuie să definească clasa din care se crează obiectul de ascultare (obiect ce se ataşează sursei de evenimente).

În această clasă de ascultare se defineşte metoda de tratare a evenimentului.

Mediul de execuţie Java este cel care detectează evenimentele şi apelează în mod automat, la detectarea evenimentului respectiv, metoda corespunzătoare de tratare a evenimentului (metodă scrisă de programator în clasa de ascultare). În plus, mediul de execuţie Java este cel care impune numele metodei de tratare a evenimentului, dar programatorul este cel care îi dă codul.

Astfel, pentru tratarea evenimentului, click cu mouse-ul pe un buton, numele impus de metoda de tratare este **actionPerformed( ) .**

Fereastra aplicaţiei extinde clasa de bibliotecă **JFrame**. De la aceasta moşteneşte forma(bară de titlu, cele 3 butoane:minimizare, maximizare şi închidere) şi comportamentul de fereastră (mărirea, micşorarea dimensiunilor ferestrei, mutarea ferestrei). Componentele grafice, în cazul utilizării ferestrei de tip **JFrame** (definită în pachetul **javax.swing**), nu se adaugă direct pe suprafaţa ferestrei, ci într-un container extras din fereastră cu ajutorul metodei **getContentPane( ).**

Din considerente de portabilitate nu se recomandă poziţionarea componentelor grafice la coordonate absolute.

Poziţionarea componentelor se face cu ajutorul unor clase denumite **layout manager**(clase ce fac gestionarea modului de poziţionare).

Astfel, containerul extras din **JFrame** are ca şi layout, mod de aranjare implicit al componentelor, layout-ul de tip **BorderLayout.**

Componentele pot fi aranjate în 5 direcţii (cele patru puncte cardinale si centrul): **North**, **South**, **East**, **West**, **Center**.

Mai sunt si alte layout-uri posibile pe lângă **BorderLayout.** De exemplu:

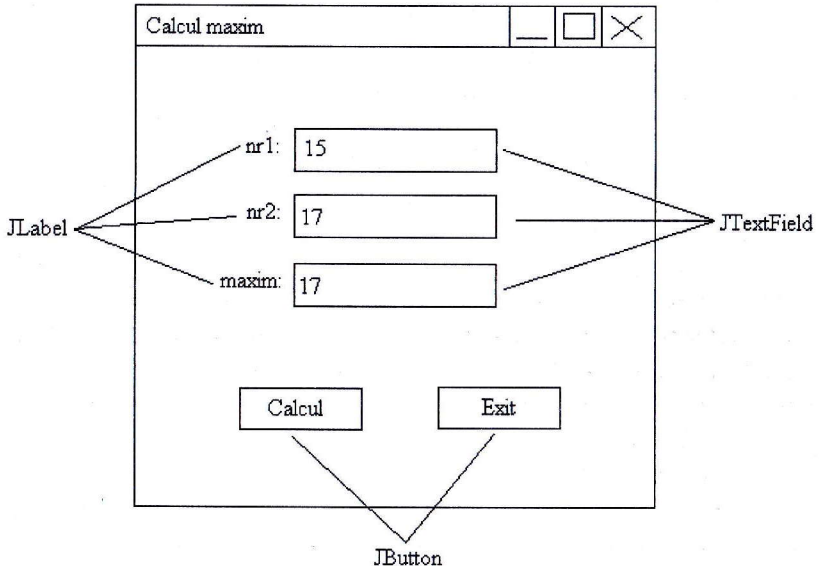
- **FlowLayout** – în care componentele grafice sunt aranjate în linie, de la stânga la dreapta. Obiectele de tip **JPanel**, au implicit acest layout.
- **GridLayout** – în care componentele grafice sunt aranjate într-o matrice, specificată prin numărul de linii şi numărul de coloane.

Layout-ul implicit al unui container poate fi modificat la un nou layout, folosind metoda **setLayout( )** care are ca parametru un obiect instanţiat din noul layout.

2. Exemple.

Exemplul 1

Scriem o aplicaţie grafică , pentru calculul maximului dintre doua numere, ce conţine următoarele componente grafice:




```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class CalculMaxim
{
    public static void main(String args[ ])
```

```
    {
        Fereastra f=new Fereastra( );
        f.setVisible(true);
    }
}
class Fereastra extends JFrame
{
    private JButton, jbCalcul, jbExit;
    private JTextField jtf1, jtf2, jtf3;
    public Fereastra( )
    {
        this.setTitle("Calcul maxim");
        this.setSize(600, 400);
        JLabel jl1=new JLabel("nr1:");
        jtf1=new JTextField(40);
        JPanel jp1=new JPanel( );
        jp1.add(jl1); jpd.add(jtf1);
        JLabel jl2=new JLabel("nr2:");
        jtf2=new JTextField(40);
        JPanel jp2=new JPanel( );
        jp2.add(jl2); jp2.add(jtf2);
        JLabel jl3=new JLabel("maxim:");
        jtf3=new JTextField(40);
        jtf3.setEditable(false);
        JPanel jp3=new JPanel( );
        jp3.add(jl3); jpd.add(jtf3);
        AscultăButoane ab=new AscultăButoane( );
        jbCalcul=new JButton("Calcul");
        jbCalcul.addActionListener(ab);
        jbExit=new JButton("Exit");
        jbExit.addActionListener(ab);
        JPanel jp4=new JPanel( );
        jp4.add(jbCalcul);
        jp4.add(jbExit);
        JPanel jp=new JPanel( );
        jp.setLayout(new GridLayout(4,1));
        Container cFinal=this.getContentPane( );
        cFinal.add(jp, "South");
    }// end constructor
/*
```


În continuare în clasa Fereastra definim ca şi clasă interioară, clasa de ascultare a butoanelor: clasa AscultăButoane. O clasă interioară a unei alte clase are acces în mod direct la toate variabilele de instanţă a unei clase private.

```
*/
  // clasa interioară:
  class AscultăButoane implements ActionListener
  {
    public void actionPerformed(ActionEvent ev)
    {
        Object sursa=ev.getSource( );
        if (sursa= =jbExit) System.exit(0);
        else if (sursa= =jbCalcul){
                String s1=jtf1.getText( );
                int nr1=Integer.parseInt(s1);
                int nr2=Integer.parseInt(jtf2.getText( ));
                int max;
                if (nr1>nr2) max=nr1;
                else max=nr2;
                jtf3.setText(""+max); }
    }//actionPerformed
  }
}
```

Exemplul 2

Să se scrie o aplicaţie ce conţine patru componente grafice: JTextField pentru afişare şi trei butoane JButton, notate cu "unu", "doi" şi "trei". Cele trei butoane vor fi aşezate în linie. La apăsarea unui buton se va afişa în componenta JTextField numele butonului apăsat.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class ScrieNumeButon
{
 public  static void main(String args[])
 {
  Fereastra f=new Fereastra();
  f.setTitle("Test de butoane");
  f.setVisible(true);
```

```
 }
}
class Fereastra extends JFrame
{
 private JTextField jtf;
 private JButton jb1, jb2, jb3;
 //constructor
 public Fereastra()
 {
  setSize(300,300);
  addWindowListener(new FereastraListener());
  //construieste componente:
  jtf=new JTextField();
  //doar pt. afisare:
  jtf.setEditable(false);
 //Construim cele trei butoane:
 jb1=new JButton("unu");
 jb2=new JButton("doi");
 jb3=new JButton("trei");
 //Le plasez intr-un container intermediar JPanel:
 JPanel jp=new JPanel();
 jp.add(jb1);jp.add(jb2);jp.add(jb3);
 //Atasam fiecarui buton, obiectul de ascultare, pentru a putea fi
 //tratate evenimentele de apasare a acestor butoane:
 ClasaButoaneListener listener=new ClasaButoaneListener();
 jb1.addActionListener(listener);
 jb2.addActionListener(listener);
 jb3.addActionListener(listener);
 //adaugam componentele in containerul final, ce se extrage din
 //fereasta:
 Container containerFinal=this.getContentPane();
 containerFinal.add(jtf,"Center"); //daca lipseste, nu deseneaza nimic
 //in centru
 containerFinal.add(jp,"South");//le aseaza in partea de jos a
 //ferestrei
}
 //clasa interioara:
 private class FereastraListener extends WindowAdapter
 {
 public void windowClosing(WindowEvent ev)
```



```
 {
  System.exit(0);
 }
}private class ClasaButoaneListener implements ActionListener
{
 //Interfata ActionListener are o singura metoda:
 public void actionPerformed(ActionEvent ev)
 {
  Object sursa=ev.getSource();
  if(sursa==jb1)jtf.setText("unu");
  else if(sursa==jb2)jtf.setText("doi");
  else if(sursa==jb3)jtf.setText("trei");
 }
}
}
```

Exemplul 3

Să se scrie o aplicație în care se afișează o fereastră ce conține patru componente grafice: JTextField pentru introducerea unui număr întreg, JTextField pentru afişare şi două componente JButton. Atunci când este apăsat primul buton, se vor afişa toți divizorii numărului introdus. Când se apasă cel de-al doilea, se iese din program.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class GraficAfisareDivizori
{
 public static void main(String args[])
 {
  Fereastra f=new Fereastra();
  f.setTitle("Calcul divizori");
  f.setVisible(true);
 }
}
class Fereastra extends JFrame
{
 private JTextField jtfDate, jtfAfisare;
 private JButton jbCalcul, jbExit;
//constructor
 public Fereastra()
```

```
{
 setSize(600,400);
 addWindowListener(new FereastraListener());
 //construieste componente:
 JLabel jl1=new JLabel("Numar=");
 jtfDate=new JTextField(5);//pt. introducere numar
 JPanel jp1=new JPanel();
 jp1.add(jl1); jp1.add(jtfDate);

 JLabel jl2=new JLabel("Divizorii: ");
 jtfAfisare=new JTextField(40);
 jtfAfisare.setEditable(false) ;
 JPanel jp2=new JPanel();
 jp2.add(jl2); jp2.add(jtfAfisare);

 jbCalcul=new JButton("Calcul");
 jbExit=new JButton("Exit");
 JPanel jp3=new JPanel();
 jp3.add(jbCalcul); jp3.add(jbExit);

 //Container intermediar:
 JPanel jp=new JPanel();
 jp.setLayout(new GridLayout(3,1));
 jp.add(jp1); jp.add(jp2); jp.add(jp3);

 ClasaButoaneListener bL=new ClasaButoaneListener();
 jbCalcul.addActionListener(bL);
 jbExit.addActionListener(bL);
 //Containerul final:
 Container c=this.getContentPane();
 c.add(jp,"South");
}
 //inner class:
 private class FereastraListener  extends WindowAdapter
 {
  public void windowClosing(WindowEvent ev)
  {
   System.exit(0);
  }
 }//end inner class
```



```
 //inner class:
 private class ClasaButoaneListener implements ActionListener
 {
  public void actionPerformed(ActionEvent ev)
  {
   Object sursa=ev.getSource();
   if(sursa==jbExit)System.exit(0);
   if(sursa==jbCalcul){
    int nr=Integer.parseInt(jtfDate.getText());
    String rezultat="1, ";//primul divizor
    for(int i=2;i<=nr/2;i++)
     if(nr%i==0)rezultat=rezultat+i+", ";
    //Ultimul divizor este chiar nr:
     rezultat=rezultat+nr+".";
     jtfAfisare.setText(rezultat);
   }
  }
 }//end inner class
}//end class Fereastra
```

Exemplul 4

Să se scrie o aplicație în care se afișează o fereastră ce conține patru componente grafice: JTextField pentru introducerea elementelor unui vector de numere intregi (elementele sunt separate prin spații), JTextField pentru afișarea maximului din vector şi două componente JButton. Atunci când este apăsat primul buton, se va afișa maximul din vectorul introdus. Când se apasă cel de-al doilea, se iese din program.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class GraficAfisareMaxim
{
public static void main(String args[])
{
 Fereastra f=new Fereastra();
 f.setTitle("Calcul maxim");
 f.setVisible(true);
}
```

```
}

class Fereastra extends JFrame
{
 private JTextField jtfDate, jtfAfisare;
 private JButton jbCalcul, jbExit;
 //constructor
 public Fereastra()
 {
  setSize(600,400);
  addWindowListener(new FereastraListener());
  //construieste componente:
  JLabel jl1=new JLabel("Vector=");
  jtfDate=new JTextField(40);//pt. introducere numar
  JPanel jp1=new JPanel();
  jp1.add(jl1); jp1.add(jtfDate);

  JLabel jl2=new JLabel("Maxim: ");
  jtfAfisare=new JTextField(5);
  jtfAfisare.setEditable(false)  ;
  JPanel jp2=new JPanel();
  jp2.add(jl2); jp2.add(jtfAfisare);

  jbCalcul=new JButton("Calcul");
  jbExit=new JButton("Exit");
  JPanel jp3=new JPanel();
  jp3.add(jbCalcul); jp3.add(jbExit);

  //Container intermediar:
  JPanel jp=new JPanel();
  jp.setLayout(new GridLayout(3,1));
  jp.add(jp1); jp.add(jp2); jp.add(jp3);


  ClasaButoaneListener bL=new ClasaButoaneListener();
  jbCalcul.addActionListener(bL);
  jbExit.addActionListener(bL);
  //Containerul final:
  Container c=this.getContentPane();
  c.add(jp,"South");
```



```
 }
 //inner class:
 private class FereastraListener  extends WindowAdapter
 {
  public void windowClosing(WindowEvent ev)
  {
   System.exit(0);
  }
 }//end inner class
 //inner class:
 private class ClasaButoaneListener implements ActionListener
 {
  public void actionPerformed(ActionEvent ev)
  {
    Object sursa=ev.getSource();
    if(sursa==jbExit)System.exit(0);
    if(sursa==jbCalcul){
          //preluarea elementelor vectorului, din JTextField:
          String s=jtfDate.getText();
          StringTokenizer tk=new StringTokenizer(s);
          int N=tk.countTokens();//nr. de elemente din vector
          //initializam maximul cu primul numar :
          int max=Integer.parseInt(tk.nextToken());
          //Comparam cu restul numerelor:
          for(int i=1;i<N;i++){
                int nrCrt=Integer.parseInt(tk.nextToken());
                if(nrCrt>max)max=nrCrt;
          }
          //Afisare maxim:
          jtfAfisare.setText(""+max);
    }
  }
 }//end inner class
}//end class Fereastra
```

Exemplul 5

Să se scrie o aplicație în care se afișează o fereastră ce conține trei componente grafice: JTextArea pentru introducerea pe mai multe linii a unor numere întregi (în fiecare linie sunt mai multe numere separate prin spații), JTextField pentru afișare și JButton pentru

comanda efectuării calculelor. Atunci când se apasă butonul, în JTextField se va afişa suma tuturor numerelor din JTextArea.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class AfisareSuma
{
 public static void main(String args[])
 {
  Fereastra f=new Fereastra();
  f.setTitle("Suma numerelor");
  f.setVisible(true);
 }
}

class Fereastra extends JFrame
{
 private JTextArea jta;
 private JTextField jtfAfisare;
 private JButton jbCalcul;
 //constructor
 public Fereastra()
 {
  setSize(600,400);
  addWindowListener(new FereastraListener());
  jta=new JTextArea();
  jtfAfisare=new JTextField(5);
  jtfAfisare.setEditable(false)  ;
  JPanel jp1=new JPanel();
  jp1.add(jtfAfisare);
  jbCalcul=new JButton("Calcul");
  JPanel jp2=new JPanel();
  jp2.add(jbCalcul);

  //Container intermediar:
  JPanel jp=new JPanel();
  jp.setLayout(new GridLayout(2,1));
  jp.add(jp1); jp.add(jp2);
```



```
  ClasaButoaneListener bL=new ClasaButoaneListener();
  jbCalcul.addActionListener(bL);
  //Containerul final:
  Container c=this.getContentPane();
  c.add(jta,"Center");
  c.add(jp,"South");
 }

 //inner class:
 private class FereastraListener  extends WindowAdapter
 {
  public void windowClosing(WindowEvent ev)
  {
   System.exit(0);
  }
 }//end inner class
 //inner class:
 private class ClasaButoaneListener implements ActionListener
 {
  public void actionPerformed(ActionEvent ev)
  {

     String s=jta.getText();
     //Extrag numerele :
     StringTokenizer tk=new StringTokenizer(s);
     int N=tk.countTokens();
     int suma=0;
     for(int i=0;i<N;i++)
       suma=suma+Integer.parseInt(tk.nextToken());
     //Afisare suma:
     jtfAfisare.setText(""+suma);
   }
 }
}//end class Fereastra
```

Exemplul 6

Să se scrie o aplicaţie în care se afişează o fereastră ce conţine trei componente grafice: JTextArea pentru introducerea unui text, JTextField pentru afişare şi JButton pentru comanda efectuării

calculelor. Atunci când se apasă butonul, în JTextField se va afişa numarul de caractere 'a' din textul introdus în JTextArea.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class Calcul
{
 public static void main(String args[])
 {
  Fereastra f=new Fereastra();
  f.setTitle("Calcul nr. caractere a");
  f.setVisible(true);
 }
}

class Fereastra extends JFrame
{
 private JTextArea jta;
 private JTextField jtfAfisare;
 private JButton jbCalcul;
 //constructor
 public Fereastra()
 {
  setSize(600,400);
  addWindowListener(new FereastraListener());
  jta=new JTextArea();
  jtfAfisare=new JTextField(5);
  jtfAfisare.setEditable(false)  ;
  JPanel jp1=new JPanel();
  jp1.add(jtfAfisare);
  jbCalcul=new JButton("Calcul");
  JPanel jp2=new JPanel();
  jp2.add(jbCalcul);
    //Container intermediar:
  JPanel jp=new JPanel();
  jp.setLayout(new GridLayout(2,1));
  jp.add(jp1); jp.add(jp2);
   ClasaButoaneListener bL=new ClasaButoaneListener();
  jbCalcul.addActionListener(bL);
```



```
  //Containerul final:
  Container c=this.getContentPane();
  c.add(jta,"Center");
  c.add(jp,"South");
 }
 //inner class:
 private class FereastraListener  extends WindowAdapter
 {
  public void windowClosing(WindowEvent ev)
  {
   System.exit(0);
  }
 }//end inner class
 //inner class:
 private class ClasaButoaneListener implements ActionListener
 {
  public void actionPerformed(ActionEvent ev)
  {
     String s=jta.getText();
     int contor=0;
     for(int i=0;i<s.length();i++)
       if(s.charAt(i)=='a')contor++;
     jtfAfisare.setText(""+contor);
   }
  }
}//end class Fereastra
```

Exemplul 7

Să se scrie o aplicaţie în care se afişează o fereastră ce conţine următoarele componente grafice: JTextField pentru afişarea textului: *Afisare cu diferite marimi de fonturi !*, trei butoane radio JRadioButton, pentru selecţia mărimii fontului cu care se scrie textul din JTextField (font mic, font mediu, font mare), şi un buton JButton pentru ieşirea din program. Atunci când este activat primul buton radio, textul se va afişa în JTextField folosind font de mărime 12, când este activat al doilea buton radio, textul se va afişa cu font de mărime 18, când este activat al treilea, se va folosi font de mărime 24.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
import java.util.*;
class GraficButoaneRadio
{
 public static void main(String args[])
 {
  Fereastra f=new Fereastra();
  f.setTitle("Butoane radio");
  f.setVisible(true);
 }
}

class Fereastra extends JFrame
{
 private JTextField jtfAfisare;
 private JButton jbExit;
 private JRadioButton radio1,radio2,radio3;

 //constructor
 public Fereastra()
 {
  setSize(600,400);
  addWindowListener(new FereastraListener());
  //construieste componente:
  jtfAfisare=new JTextField
("Afisare cu diferite marimi de fonturi !");
  jtfAfisare.setEditable(false)  ;
   radio1= new JRadioButton("font mic");
  radio1.setSelected(true);
  radio2= new JRadioButton("font mediu");
  radio2.setSelected(false);
  radio3= new JRadioButton("font mare");
  radio3.setSelected(false);
  ButtonGroup radioGrup= new ButtonGroup();
  radioGrup.add(radio1);
  radioGrup.add(radio2);
  radioGrup.add(radio3);
  JPanel jpRadio= new JPanel();
  //in JPanel nu este permisa adaugarea de ButtonGroup !
  jpRadio.add(radio1);
  jpRadio.add(radio2);
```



```
  jpRadio.add(radio3);

  jbExit=new JButton("Exit");
  JPanel jpExit=new JPanel();
  jpExit.add(jbExit);

  //Container intermediar:
  JPanel jp=new JPanel();
  jp.setLayout(new GridLayout(2,1));
  jp.add(jpRadio); jp.add(jpExit);

  ClasaButoaneListener bL=new ClasaButoaneListener();
  jbExit.addActionListener(bL);
  radio1.addActionListener(bL);
  radio2.addActionListener(bL);
  radio3.addActionListener(bL);
  //Containerul final:
  Container c=this.getContentPane();
  c.add(jtfAfisare,"Center");
  c.add(jp,"South");
  //Scriem textul in jtfAfisare cu font mic (marimea 12):
  jtfAfisare.setFont(new Font("MonoSpaced",Font.ITALIC,12));
  jtfAfisare.repaint();
 }
 //inner class:
 private class FereastraListener  extends WindowAdapter
 {
  public void windowClosing(WindowEvent ev)
  {
   System.exit(0);
  }
 }//end inner class
 //inner class:
 private class ClasaButoaneListener implements ActionListener
 {
  public void actionPerformed(ActionEvent ev)
  {
    Object sursa=ev.getSource();
    if(sursa==jbExit)System.exit(0);
    else{
```

```
    String tipFont="MonoSpaced";
    int stilFont=Font.ITALIC;
    int marimeFont=12;
    if(radio2.isSelected())marimeFont=18;
    else if(radio3.isSelected())marimeFont=24;
    jtfAfisare.setFont(new Font(tipFont,stilFont,marimeFont));
    jtfAfisare.repaint();
   }
  }
 }
}//end class Fereastra
```

Exemplul 8

Să se scrie o aplicație în care se afișează o fereastră ce conține trei componente grafice: JTextField pentru introducerea unui text, JTextField pentru afișare și un buton JButton pentru ieșirea din program. Atunci când se apasă tasta ENTER în primul JTextField, textul introdus acolo va fi copiat în al doilea JTextField.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
class GraficCopiereText
{
 public static void main(String args[])
 {
  Fereastra f=new Fereastra();
 f.setTitle("Copiere text");
 f.setVisible(true);
 }
}

class Fereastra extends JFrame
{
 private JTextField jtfDate,jtfAfisare;
 private JButton jbExit;

 //constructor
 public Fereastra()
 {
```



```
 setSize(600,400);
 addWindowListener(new FereastraListener());
 //construieste componente:
 jtfDate=new JTextField(40);
 jtfDate.setEditable(true);
 jtfDate.addKeyListener(new ClasaTasteListener());
 JPanel jp1=new JPanel();
 jp1.add(jtfDate);

 jtfAfisare=new JTextField(40);
 jtfAfisare.setEditable(false);
 JPanel jp2=new JPanel();
 jp2.add(jtfAfisare);

 jbExit=new JButton("Exit");
 JPanel jp3=new JPanel();
 jp3.add(jbExit);

 //Container intermediar:
 JPanel jp=new JPanel();
 jp.setLayout(new GridLayout(3,1));
 jp.add(jp1); jp.add(jp2); jp.add(jp3);

 ClasaButoaneListener bL=new ClasaButoaneListener();
 jbExit.addActionListener(bL);
 //Containerul final:
 Container c=this.getContentPane();
 c.add(jp,"South");
}
//inner class:
private class FereastraListener  extends WindowAdapter
{
 public void windowClosing(WindowEvent ev)
 {
  System.exit(0);
 }
}//end inner class
//inner class:
private class ClasaButoaneListener implements ActionListener
{
```

```
 public void actionPerformed(ActionEvent ev)
 {
  System.exit(0);
 }
 }

 private class ClasaTasteListener extends KeyAdapter
 {
 public void keyPressed(KeyEvent evt)
 {
  // S-a tastat ENTER ?
  if (evt.getKeyChar() == '\n') {
   jtfAfisare.setText(jtfDate.getText());
  }
  }
 }
}//end class Fereastra
```

Bibliografie

1. Georgescu H., Introducere in universul Java, Ed. Tehnica, 2002
2. Keogh J., Java fara mistere, Ed. Rosetti Educational, 2006
3. Tanasa S., Olaru C., Andrei S. – Java de la 0 la expert, Ed. Polirom, 2003
4. Ene Al.- Programare orientată pe obiecte, note de curs, Universitatea din Piteşti, 2007
5. Ene Al., Stirbu C. – 100 de probleme rezolvate in Java, Ed. Universitatii din Pitesti, 2007

ISBN 978-973-690-845-3